Ukrainian Catholic University

Bachelor Thesis

CXU Playground: RISC-V Custom Extension Development Framework

Author:
Petro Mozil

Supervisor:
Dr. Oleh Farenyuk

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences

Lviv 2026

"*If you wish to make an apple pie from scratch, you must first invent the universe.*"

Carl Sagan

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

CXU Playground: RISC-V Custom Extension Development Framework

by Petro Mozil

Abstract

Custom hardware extensions for RISC-V processors can be a way to gain significant performance for compute-bound or compute- and memory-bound workloads, such as cryptography, data compression, and machine learning inference. The Composable eXtension Unit (CXU) specification was proposed as a modern, flexible interface for integrating multiple custom extensions into a RISC-V processor, supporting shared extension state and higher-level language integration.

At the time of writing, no hardware implementation of the CXU interface exists for any RISC-V processor, and no sandbox environment supports its development and evaluation under realistic system conditions.

In this thesis we implement the CXU interface as a SpinalHDL plugin for the VexiiRiscv processor and build the CXU Playground – an open-source, full-stack development and benchmarking environment built on the LiteX SoC framework. The Playground provides automated SoC generation, a Buildroot Linux environment for OS-level testing, and a library for invoking custom extensions without inline assembly.

To validate the framework, we implement and benchmark three case-study extensions: an AES GF(2^8) accelerator, a GZIP/DEFLATE bit-manipulation accelerator, and a multiply-accumulate (MAC) extension for TensorFlow Lite Micro inference.

The AES extension achieves a consistent 5.25× reduction in cycles per encrypted byte, confirming that tightly coupled custom instructions can yield substantial gains for arithmetic-intensive embedded workloads.

Acknowledgements

Firstly, I would like to thank the Ukrainian Armed Forces for their insurmountable courage and sacrifice. It is thanks to them that we are able to live, study, and work in independent Ukraine.

I am deeply thankful to my supervisor, Oleg Farenyuk, for his continued support, guidance, and help. Thank you for your guidance in refining the academic rigor of this work and for steering the project toward outcomes with practical relevance beyond academia.

I am also deeply grateful to my friends and family, who supported me even in times when I did not have faith.

Contents

Abstract **ii**

Acknowledgements **iii**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Contributions . . . 1
1.3 Structure Of The Thesis . . . 2

2 Related Work **3**
2.1 RISC-V Extensions For Custom Instruction Integration . . . 3
2.1.1 CFU . . . 3
2.1.2 CXU . . . 4
2.1.3 Core-V-XIF . . . 4
2.2 Frameworks for FPGA-based SOC Development . . . 4
2.2.1 LiteX . . . 4
2.2.2 F4PGA . . . 5
2.2.3 MiSoC . . . 5
2.2.4 PULP Platform . . . 5
2.2.5 SpinalHDL . . . 5
2.3 Sandboxes for Simplified Development of Custom Instruction Hardware 5
2.3.1 FRANCIS-V . . . 5
2.3.2 CFU Playground . . . 6
2.4 Applications of Custom Instruction Extensions . . . 7
2.4.1 AES and Symmetric Cryptography . . . 7
2.4.2 Post-Quantum Cryptography . . . 7
2.4.3 Hashing . . . 7
2.4.4 Data Compression . . . 8
2.4.5 Machine Learning Inference . . . 8
2.4.6 Linux on LiteX . . . 8

3 Theoretical Background **9**
3.1 The RISC-V Architecture and VexiiRiscv Core . . . 9
3.1.1 Overview of the RISC-V ISA . . . 9
3.1.2 The VexiiRiscv Implementation . . . 9
3.2 The LiteX SoC Ecosystem . . . 10
3.2.1 LiteX Framework Overview . . . 10
3.2.2 Memory-Mapped Architecture . . . 10
3.2.3 Interconnect and Control . . . 10
3.2.4 Memory Management . . . 11
3.3 Software Development for LiteX-Based SoCs . . . 11
3.3.1 Cross-Compilation . . . 11
3.3.2 Linker Scripts . . . 11

3.3.3 Bare-Metal vs Linux-Based Systems . . . 12
3.4 Buildroot . . . 12
3.5 Custom Extension Units (CXU) Mechanism . . . 12
3.5.1 Coupling Architecture . . . 12
3.5.2 VexiiRiscv CXU User Plugin Interface . . . 13
Instruction Decoding . . . 13
3.6 Linux Support on LiteX . . . 13
3.6.1 Hardware Requirements . . . 14
3.6.2 The Boot Process . . . 14
3.6.3 Device Tree Overlay (DTS) . . . 14
3.7 The AES Encryption . . . 15
3.7.1 AES Round Operations . . . 16
KeyExpansion . . . 16
AddRoundKey . . . 16
MixColumns . . . 16
3.7.2 Speeding up AES with the CXU . . . 16
3.8 GZIP . . . 16
3.8.1 The DEFLATE Algorithm . . . 16
3.8.2 LZ77 Compression . . . 17
3.8.3 Huffman Coding and Tree Modes . . . 17
3.8.4 Custom Instructions for DEFLATE Acceleration . . . 18
Bit Reversal . . . 18
Bit Mask Generation . . . 18
Huffman Table Index Extraction . . . 18
LZ77 Back-Reference Address Calculation . . . 18
Implementation Characteristics . . . 19
3.9 TensorFlow Lite Micro . . . 19
3.9.1 Multiply-Accumulate Operations . . . 19

4 Proposed Solution **20**
4.1 Problem Statement . . . 20
4.2 System Architecture and Playground Design . . . 20
4.3 CXU Integration in VexiiRiscv . . . 21
4.4 Sandbox Development Environment and Buildroot Integration . . . 21
4.5 The CXU Runtime . . . 22
4.6 The CXU Playground . . . 22
4.6.1 The CXU interface modification . . . 23
4.6.2 The LiteX SoC development environment . . . 23
4.6.3 The environment for bare-metal application development . . . 23
4.6.4 The AES implementation . . . 23
4.6.5 The GZIP implementation . . . 23
4.6.6 TensorFlow Lite Micro MAC speedup . . . 23
4.7 Validation through Case Studies . . . 24
4.8 Testing and Benchmarking Methodology . . . 24
4.9 Data Analysis . . . 25
4.10 Target Hardware: Digilent Arty S7-50 . . . 25
4.11 Limitations . . . 25
4.11.1 CXU CSR state IO /switch latency . . . 25
4.11.2 CXU Capabilities in the Linux Device Tree . . . 26
4.11.3 Clock Frequency Limitations . . . 26
4.11.4 Multi-Process CXU Access and Kernel Driver . . . 26

5 Experiments and Results **27**
5.1 Experiments and Benchmarks 27
5.1.1 AES $GF(2^8)$ extension and software comparison 27
5.1.2 GZIP extension and software implementation 28
5.1.3 TensorFlow MAC extension 30

6 Conclusions **31**
6.1 Summary of Contributions 31
6.2 Key Results 31
6.3 Limitations of the Current Implementation 32
6.4 Future Work 32
6.5 Closing Remarks 33

A Detailed FPGA Synthesis Area Reports **34**

Bibliography **38**

List of Figures

2.1 FRANCIS-V framework overview. 6
2.2 CFU Playground full-stack architecture. 6

3.1 Hardware architecture of the generated LiteX SoC. 11
3.2 Logical interface between the VexiiRiscv CPU and a CXU accelerator. 13
3.3 The LiteX SoC Linux boot process. 14
3.4 Flowchart of the AES encryption algorithm. 15
3.5 Flowchart of the DEFLATE compression pipeline. 17

4.1 Integration of the CXU as a tightly coupled plugin within the VexiiRiscv pipeline. 21
4.2 The CXU Sandbox full-system stack. 22

5.1 Runtime comparison of GZIP runtime for different file sizes. 29

List of Tables

5.1 Runtime comparison of AES runtime for different buffer sizes. 27
5.2 Throughput comparison of AES runtime for different buffer sizes. . . . 28
5.3 Inference latency comparison for TFLite Micro using the CXU MAC extension. 30

A.1 Per-module cell counts for the AES CXU configuration (total: 20 948 cells). 35
A.2 Per-module cell counts for the GZIP CXU configuration (total: 20 666 cells). 36
A.3 Per-module cell counts for the TFLite MAC CXU configuration (total: 20 698 cells). There is no separate `Cxu0` top-level instance; the MAC unit is optimized out for this specific instance. 37

List of Abbreviations

ABI	**A**pplication **B**inary **I**nterface
AES	**A**dvanced **E**ncryption **S**tandard
CFU	**C**omposable **F**unction **U**nit
CPU	**C**entral **P**rocessing **U**nit
CSR	**C**ontrol/**S**tatus **R**egister
CXU	**C**omposable **X**tension **U**nit
DTB	**D**evice **T**ree **B**inary
DTS	**D**evice **T**ree **S**ource
MMU	**M**emory **M**anagement **U**nit
PQC	**P**ost **Q**uantum **C**ryptography
RVTSO	**R**ISC-**V** **T**otal **S**tore **O**rdering
RVWMO	**R**ISC-**V** **W**eak **M**emory **O**rdering
SBI	**S**upervisor **B**inary **I**nterface
SoC	**S**ystem **o**n **C**hip
XLEN	The length of a RISC-V general-purpose register in bits fo a given CPU

Dedicated to my friends and family - for your love and faith

Chapter 1

Introduction

1.1 Motivation

RISC-V [32] is a modern and extensible instruction set architecture often used in systems-on-chip, especially FPGA-based ones [10], such that it is both architecturally simple and energy-efficient. RISC-V's instruction set architecture is designed with custom extensions in mind, so designing custom extensions would be a straightforward process. It is, however, a difficult task to have a custom extension be part of the RISC-V specification (the ratification process involves a big part of the community voting on the details of the specification), and even more so to implement an extension for the multitude of RISC-V processor implementations. With such issues, and the fact that most FPGA-based SOCs offer plenty of FPGA frabric not used by the SoC itself to also implement an extension, in mind, Jan Gray et al. designed the custom composable extensions (CFU) extension [16]. It allows for an implementation of a specific extension to be used across multiple RISC-V CPUs, so as to use the single extension implementation across a multitude of different RISC-V CPUs. CFU allows for a single extension to be connected to a hardware thread. It also allows passing a "function index" to an extension, such that up to 255 different functions may be implemented.

There is also a playground for designing the CFU extensions on a ready-made SOC called the CFU playground. It allows the users to implement a CFU and write a main script that runs and invokes the implemented extension. CFU is a useful interface for implementing a single extension, but when combining extensions, especially those that may have multiple different functions and utilize the function ID, managing them becomes complex and introduces significant software overhead. Because of that, Gray et al. also designed a newer version of composable custom extensions – the CXU [17]. This implementation allows for switching between multiple custom extensions, but, as of the time of writing, there is not yet an implementation of it on any of the RISC-V implementations, nor an environment for developing custom extensions.

1.2 Contributions

This thesis makes the following concrete contributions:

- **First CXU implementation:** A SpinalHDL plugin for the VexiiRiscv processor that implements the CXU interface, including opcode interception at the execute stage, pipeline stall management for multi-cycle operations, and `cxidx`/`cxdata` CSR support.
- **CXU Playground:** A unified, open-source sandbox built on LiteX that automates SoC generation, Buildroot Linux integration, cross-compilation, and CXU

hardware templating, enabling developers to prototype and benchmark custom extensions under realistic, OS-level conditions.

- **CXU runtime library:** A header-only library that wraps the low-level assembly invocation, manages extension multiplexing via the `cxidx` register, and provides a portable C API for CXU-accelerated routines.
- **Three case-study extensions:** An AES GF(2^8) accelerator, a GZIP/DEFLATE bit-manipulation accelerator, and a multiply-accumulate (MAC) extension for TensorFlow Lite Micro inference, each accompanied by a paired software reference implementation and benchmarking suite.

Benchmarking results demonstrate approximately a 5.25× reduction in cycles per byte for AES encryption and a roughly 10% reduction in CPU time for GZIP bitstream manipulation, confirming that even a small number of tightly coupled custom instructions can yield substantial gains for arithmetic-intensive workloads (see chapter 5).

1.3 Structure Of The Thesis

- chapter 2 surveys existing custom-instruction interfaces (CFU, CXU, Core-V-XIF), FPGA-based SoC frameworks, related sandboxes, and prior work applying custom ISA extensions to cryptography, compression, and machine learning.
- chapter 3 provides the theoretical background covering the RISC-V ISA, the VexiiRiscv pipeline, the LiteX SoC ecosystem, the CXU coupling mechanism, Linux boot flow on LiteX, and the algorithmic foundations of the three case studies.
- chapter 4 describes the design of the CXU Playground: the system architecture, the CXU plugin integration in VexiiRiscv, the Buildroot environment, the runtime library, and the benchmarking methodology.
- chapter 5 presents the implementation details and experimental results for the AES, GZIP, and TFLite Micro extensions, and discusses unimplemented features and known limitations.
- chapter 6 summarises the contributions, reflects on the limitations of the current implementation, and outlines directions for future work.

Chapter 2

Related Work

The growing interest in domain-specific hardware acceleration on RISC-V processors has produced a range of interface standards, development frameworks, and sandboxes, each addressing part of the problem this thesis targets. This chapter surveys the most relevant prior work across three dimensions: the interface specifications that define how custom extensions attach to a RISC-V core (Section 2.1), the SoC frameworks used to build the surrounding hardware platform (Section 2.2), and the sandboxes designed to lower the barrier to prototyping custom instructions (Section 2.3). The chapter closes with a survey of application domains — cryptography, compression, and machine learning — where prior work has demonstrated the practical value of tightly coupled custom instructions, establishing the benchmarks against which the extensions developed in this work are evaluated (Section 2.4).

2.1 RISC-V Extensions For Custom Instruction Integration

Several interface specifications have been proposed to standardize the connection between a RISC-V processor and custom hardware accelerators. These specifications differ considerably in the number of extensions they support, the degree of OS integration they define, and the constraints they impose on the programmer, all of which directly motivated the design choices made in this work.

2.1.1 CFU

The Custom Function Unit (CFU) specification [16, 17, 18] was proposed by Jan Gray et al. to make custom extensions modular and reusable across different RISC-V CPU implementations. The motivation was straightforward: many RISC-V processors incorporate some form of custom extension — for instance, a SIMD multiply-accumulate unit — yet no cross-implementation interoperability exists, resulting in duplicated effort whenever the same acceleration is ported to a new core. The CFU addresses this by defining a common logic interface to which a single custom extension can be attached, along with timing, logical, and combinatorial constraints. The full interface specification is examined in Chapter 3.

Despite its value in establishing the concept of composable extensions, the CFU has important limitations. Only one extension may be connected per hardware thread; the extension may not access memory, and even when programming in a higher-level language, the user must invoke a **custom-0|1|2|3** assembly instruction directly [16]. These constraints make the CFU impractical for workloads that require multiple distinct accelerators or expect a clean programming model.

2.1.2 CXU

The CXU specification [17] is the successor to the CFU, designed to address its principal limitations. It supports up to $2^{\mathbf{XLEN}}$, where **XLEN** is the number of bits in it's general-purpose registers, simultaneous extensions, permits extensions to maintain a shared state, and defines the conditions under which an extension may access memory. Notably, the CXU specification also defines a standardized Application Binary Interface (ABI), making it possible to invoke an extension from C without inline assembly. A reference runtime implementation is provided, in which calls to CXU extensions are wrapped in helper functions that select the appropriate extension via the `cxidx` CSR, save the previous state, and clean up the context afterward. The specification was developed with operating system support in mind and explicitly defines how the kernel must save and restore CXU state during context switches. As noted in Chapter 1, to the best of authors' knowledge, no hardware implementation of this interface existed prior to the present work, which forms the primary gap this thesis fills.

2.1.3 Core-V-XIF

The **Core-V** E**X**tensible **I**nter**F**ace [25] is an alternative RISC-V extension interface that supports custom ISA extensions and coprocessors. Like the composable custom extensions family, Core-V-XIF provides a bus-level interface that any compliant CPU can expose. Unlike the CFU and CXU, it permits custom CSRs and imposes fewer restrictions on extension behaviour, offering greater flexibility at the cost of a less uniform programming model. A sandbox built on Core-V-XIF, FRANCIS-V [6], is discussed further in Section 2.3; however, because FRANCIS-V is tightly coupled to a specific CPU implementation, extensions developed for it cannot be straightforwardly ported to other cores, which limits the reusability that of the CXU specification was designed to provide.

2.2 Frameworks for FPGA-based SOC Development

Beyond the interface specification, building a CXU-capable system requires a framework that can assemble the CPU, memory, peripherals, and custom hardware logic into a coherent SoC. This section surveys the most relevant frameworks, noting for each where they fall short of the needs of this work.

2.2.1 LiteX

LiteX [10] is an open-source, Python-based framework for building FPGA-based SoCs. It provides modules for bus integration, I/O, and CPU instantiation — notably including VexiiRiscv [28] — as well as drivers for the most common FPGA development board peripherals: DDR memory via LiteDRAM, Ethernet via LiteEth, PCIE, SATA, SD cards, and standard serial buses (I2C, UART, SPI, USB). The most widely used CFU development environment, the CFU Playground [29], was built on LiteX, demonstrating its suitability as a base for custom-extension sandboxes. The present work adopts LiteX as its SoC foundation for the same reason: it abstracts low-level hardware connectivity and provides a unified build system, allowing the development effort to focus on the CXU integration itself rather than peripheral plumbing.

2.2.2 F4PGA

F4PGA [12] provides a fully open-source FPGA toolchain targeting multiple FPGA families. While not a SoC framework by itself, it is commonly paired with LiteX or custom SoC designs to enable reproducible, vendor-independent bitstream generation. It was considered as a build target for the CXU Playground but was not adopted in the final implementation, as the FPGA boards used in this work are adequately supported by vendor toolchains.

2.2.3 MiSoC

MiSoC [2] is the predecessor of LiteX and introduced the concept of Python-based SoC generation. Although it is now deprecated, it is historically significant as the origin of the design patterns that LiteX inherited and extended, and is mentioned here to provide context for the LiteX architecture described in Chapter 3.

2.2.4 PULP Platform

PULP [33] is an open-source RISC-V-based SoC platform designed for energy-efficient parallel processing and accelerator integration. Its cluster-based architecture has made it widely used in academic research on DSP, cryptography, and accelerator-centric SoC exploration, including the machine learning inference extensions discussed in Section 2.4.5. PULP was not selected as a basis for this work because its multi-core cluster model is orthogonal to the single-core, tightly coupled CXU interface that is the subject of this thesis.

2.2.5 SpinalHDL

SpinalHDL [27] is a hardware construction language, built on Scala, commonly used to implement extensible SoCs. Its plugin-based CPU architecture enables fine-grained custom instruction and pipeline extensions, making it directly relevant to composable extension research. VexiiRiscv [28] — the CPU core used in this work — is implemented in SpinalHDL, and the CXU plugin described in Chapter 4 was developed within this framework. The SpinalHDL programming model is introduced in Section 3.1.2.

2.3 Sandboxes for Simplified Development of Custom Instruction Hardware

The interface specifications and SoC frameworks described above address the infrastructure problem, but they do not by themselves lower the barrier to writing, testing, and benchmarking a custom extension. Two sandboxes have been proposed for this purpose, and both directly influenced the design of the CXU Playground developed in this work.

2.3.1 FRANCIS-V

FRANCIS-V [6] (FRAmework for iNtegrating Custom InstructionS into RISC-V systems) is a framework aimed at making the design and integration of custom extensions and coprocessors more tractable. It provides an interface and methodology for connecting custom instruction hardware to a processor using the Core-V-XIF extension, and automates simulation, compilation, and verification workflows. In these respects it shares the same high-level goal as the CXU Playground.

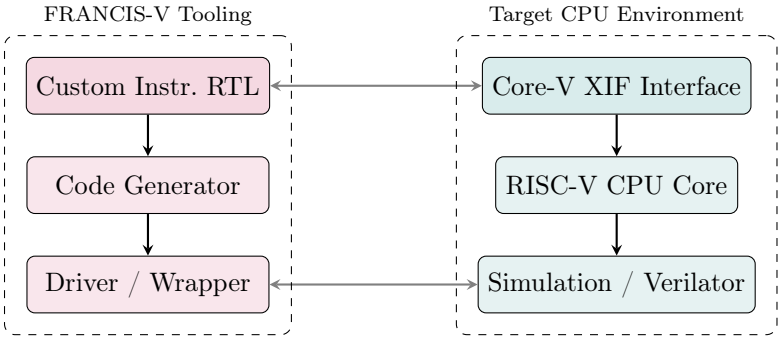

FIGURE 2.1: FRANCIS-V framework overview.

However, FRANCIS-V is tightly coupled to the Core-V-XIF interface and to the specific CPU implementations that expose it. Extensions developed within FRANCIS-V cannot be straightforwardly ported to other RISC-V cores, which conflicts with the portability goal of the composable extensions family. The CXU Playground targets the CXU interface instead, which is explicitly designed for cross-implementation reuse.

2.3.2 CFU Playground

The CFU Playground [29] is the closest prior work to the system developed in this thesis. It is a full-stack, open-source environment for the rapid design and evaluation of Custom Function Units on FPGA-based RISC-V soft CPUs, built on the LiteX framework and originally proposed for Tiny Machine Learning (tinyML) acceleration. The playground provides a ready-made SoC environment in which users can implement specialised hardware logic and write software to invoke their extensions.

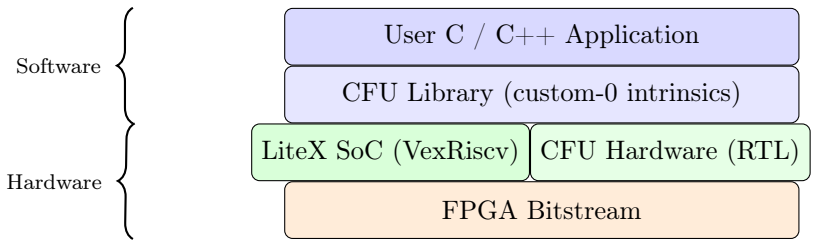

FIGURE 2.2: CFU Playground full-stack architecture.

The CFU Playground is built on the initial version of the composable custom extensions specification, which restricts each thread to a single extension and requires assembly-level **custom-0|1|2|3** opcodes even in higher-level language contexts [16, 29]. It does not support OS-level benchmarking, nor multi-process sharing (unless done cooperatively, e. g. having a semaphore for the extension), as it runs bare-metal software only. These restrictions — a single extension per thread, no memory access, no Linux support — were precisely those that motivated the CXU specification. The CXU Playground developed in this work can therefore be understood as the natural successor to the CFU Playground: it adopts the same LiteX-based architecture but realises the more capable CXU interface and adds full Linux support, enabling the system-level evaluation that the CFU Playground cannot provide.

2.4 Applications of Custom Instruction Extensions

The interface standards and sandboxes above exist to serve a practical need: a wide range of computationally intensive algorithms benefit substantially from narrowly scoped hardware acceleration. This section surveys the application domains most relevant to the case studies developed in this work — cryptography, data compression, and machine learning — and identifies the results against which the CXU extensions are compared in Chapter 5.

2.4.1 AES and Symmetric Cryptography

AES [23] is based on a substitution-permutation network whose inner operations: SubBytes (an S-box lookup over $GF(2^8)$) and MixColumns (matrix multiplication over $GF(2^8)$) — map naturally onto combinational hardware. Cui and Balasch [3] demonstrate this concretely in the context of masked AES, introducing constant-time finite-field instructions that simultaneously improve performance and resistance to side-channel attacks. Their result is particularly relevant here because it shows that a small number of carefully scoped ISA extensions — rather than a monolithic AES coprocessor — can yield substantial gains while preserving software flexibility.

Primas et al. [30] take a related approach with the NLU-V extension, fusing S-box substitution and linear diffusion into a single hardware block to minimise intermediate data movement for both AES and PRESENT. Both works support the design choice made in Chapter 4 to expose $GF(2^8)$ arithmetic as a primitive CXU instruction rather than implementing a full AES round in hardware.

2.4.2 Post-Quantum Cryptography

Classical public-key cryptosystems are vulnerable to Shor's algorithm on sufficiently powerful quantum computers, motivating the NIST post-quantum standardisation effort [36, 20]. Lattice-based schemes such as CRYSTALS-Kyber and NewHope rely on polynomial arithmetic, particularly the Number Theoretic Transform (NTT), whose butterfly operations and modular reductions are expensive on a standard ALU.

Sozbilir et al. [34] prototype NTT-specific ISA extensions on VexRiscv and report up to an 85% speedup for Kyber and NewHope, demonstrating that the class of processor used in this work is a viable target for post-quantum acceleration. Levi et al. [20] extend this further with RISQ-V, achieving a 9.6× speedup for Kyber using 29 tightly coupled custom instructions. Vranken et al. [36] argue for a uniform set of multiply-accumulate and modular-reduction primitives capable of serving both classical and post-quantum schemes, which directly foreshadows the approach taken for the MAC extension described in Section 4.7. These results collectively motivate extending the CXU Playground to post-quantum workloads as a natural direction for future work, discussed further in Section 6.4.

2.4.3 Hashing

Alawadi et al. [1] implement the full Keccak-f permutation as a single custom RISC-V instruction, reporting substantial reductions in instruction count and cycle time for SHA-3. Their area-performance trade-off analysis provides a useful reference for understanding the cost of integrating high-complexity primitives into the CXU pipeline.

2.4.4 Data Compression

GZIP/DEFLATE combines LZ77 back-reference encoding with Huffman entropy coding [35]. The performance bottleneck in decompression lies in the inner loop, which requires repeated bit-field extractions, variable-width mask generation, and modular address arithmetic — all operations that reduce to a handful of hardware gates but require several ALU instructions in software. The custom instructions designed in Section 3.8.4 and evaluated in Chapter 5 target exactly this hot path. To the best of the authors' knowledge, no prior work has applied the CXU or CFU mechanism to GZIP acceleration, making this a novel case study.

2.4.5 Machine Learning Inference

Neural network inference on microcontrollers is dominated by Multiply-Accumulate (MAC) operations in convolutional and fully connected layers. TensorFlow Lite Micro [4] provides portable INT8-quantised inference kernels for this class of device. The CMSIS-NN library [19] shows that ARM DSP SIMD MAC instructions yield up to a 4.6× improvement over scalar code; the PULP platform [33] achieves comparable gains on RISC-V with hardware-loop and dot-product extensions. The MAC extension evaluated in Chapter 5 follows the same principle, exposing a fused multiply-accumulate operation as a single-cycle CXU instruction. Oliveira et al. [24] push this further with float8 dot-product accumulation for low-precision training, demonstrating that the same CXU-style integration pattern scales to more exotic arithmetic formats.

2.4.6 Linux on LiteX

Running benchmarks inside a full operating system yields performance numbers that reflect real-world deployment conditions rather than isolated microbenchmarks. The Linux-on-LiteX-VexRiscv project [11] establishes the feasibility of booting Linux on a soft RISC-V core in a LiteX SoC, and documents the required hardware configuration: MMU support, atomic instructions, and supervisor privilege mode. This work adopts the same configuration and extends it with the CXU hardware, as described in Section 4.4.

While prior work has demonstrated the value of custom instructions for AES, post-quantum cryptography, hashing, machine learning, and (to a lesser extent) compression, none has done so within the CXU framework, and none has provided a unified sandbox environment that supports multiple extensions under Linux. The present work addresses both gaps simultaneously.

Chapter 3

Theoretical Background

This chapter provides the technical background required to understand the design decisions made in Chapters 4 and 5. It begins with the RISC-V ISA and the VexiiRiscv processor core that serves as the execution target (Section 3.1), then describes the LiteX SoC framework used to assemble the surrounding hardware platform (Sections 3.2–3.4). Section 3.5 examines the CXU coupling architecture and the VexiiRiscv plugin interface through which the extension is implemented. Section 3.6 covers the requirements and boot flow for running Linux on the generated SoC. The chapter closes with the algorithmic background for the three case-study workloads: AES encryption (Section 3.7), GZIP/DEFLATE compression (Section 3.8), and TensorFlow Lite Micro inference (Section 3.9).

3.1 The RISC-V Architecture and VexiiRiscv Core

3.1.1 Overview of the RISC-V ISA

The RISC-V instruction set architecture (ISA) [32] is an open standard designed with simplicity, extensibility, and long-term sustainability in mind. Unlike proprietary architectures such as x86 or ARM, RISC-V is developed under an open governance model and can be freely implemented without licensing constraints.

A central design principle of RISC-V is modularity. The base ISA provides only essential functionality, while optional extensions add specialized capabilities. Common extension subsets include integer multiplication (**IM**), atomic operations (**A**), single- and double-precision floating-point arithmetic (**F**/**D**), and compressed 16-bit instructions (**C**). Typical Linux-capable systems use configurations such as RV64IMAC [31]; this work uses RV32IMAFC, as required by the Linux-on-LiteX platform [11], specifically for the integer and atomic instructions used for the synchronization primitives created in linux, as well as floating point operations to simplify compiling some libraries used in this work.

Notably for this thesis, RISC-V reserves opcode space for custom extensions. These custom instruction fields allow designers to introduce application-specific accelerators directly into the processor pipeline without conflicting with standard instructions, and form the opcode basis for the CXU interface described in Section 3.5.

3.1.2 The VexiiRiscv Implementation

VexiiRiscv [28] is a configurable RISC-V core implemented in SpinalHDL [27], a Scala-based hardware description language. Unlike traditional Verilog or VHDL, SpinalHDL supports generative and highly parameterized hardware construction, enabling scalable design-space exploration, easier feature composition, and cleaner abstraction boundaries.

The processor follows a classical pipelined architecture with up to five stages: **Fetch**, **Decode**, **Execute**, **Memory**, and **Write-back**, where Fetch and Execute may be unified, and Memory and Write-back may also be unified. One of the distinguishing characteristics of VexiiRiscv is its plugin-based architecture: major features — including memory management units (MMU), caches, branch predictors, and custom execution units — are implemented as independent plugins that can be composed at generation time. These plugins are created as simple logic modules, that are enabled after some event. They are the connected to the overall logic, and triggered when necessary. The CXU extension is implemented as an execute-stage plugin; all computation is performed by CXU coprocessors and therefore occurs in the Execute stage of the pipeline, as described further in Section 3.5.

3.2 The LiteX SoC Ecosystem

Having established the processor core, this section describes the SoC framework that surrounds it. LiteX provides the integration layer between VexiiRiscv and the FPGA peripherals, and its memory-mapped architecture underpins the bus transactions used by the CXU's CSR interface.

3.2.1 LiteX Framework Overview

LiteX [10] is a Python-based SoC construction framework built on top of Migen [2]. It enables automated generation of FPGA-based SoCs by integrating CPU cores, memory controllers, interconnects, and peripherals into a cohesive hardware platform. Within this ecosystem, LiteX acts as the integration layer between the VexiiRiscv CPU core and FPGA peripherals such as UART, Ethernet [8], timers, and external memory controllers [7, 9]. The framework abstracts low-level hardware connectivity, automatically generates HDL, memory maps, CSR definitions, and software headers, and provides a unified build system for generating bitstreams and associated firmware.

3.2.2 Memory-Mapped Architecture

LiteX uses memory-mapped I/O as its primary software-hardware interface: peripherals are accessed by reading or writing specific addresses, such as:

```
#define UART_BASE 0xF0001000
```

Software interacts with hardware exclusively by manipulating these addresses. Figure 3.1 illustrates the resulting hardware architecture of the generated SoC, showing the VexiiRiscv CPU (with CXU Plugin) and the LiteEth DMA controller as bus masters, connected via the Wishbone interconnect to memory and peripheral slaves.

3.2.3 Interconnect and Control

LiteX uses the Wishbone bus as its on-chip interconnect standard. Wishbone is a simple, flexible, synchronous bus protocol that supports master-slave communication between cores and peripherals, enabling memory-mapped I/O and simplifying peripheral integration [10].

Control and Status Registers (CSRs) provide the primary mechanism for software-hardware communication. They are used for overal communication with the CPU (as, for example, for reading a timer register), and for interaction with the CXU state. CSRs are memory-mapped registers that allow software to configure hardware

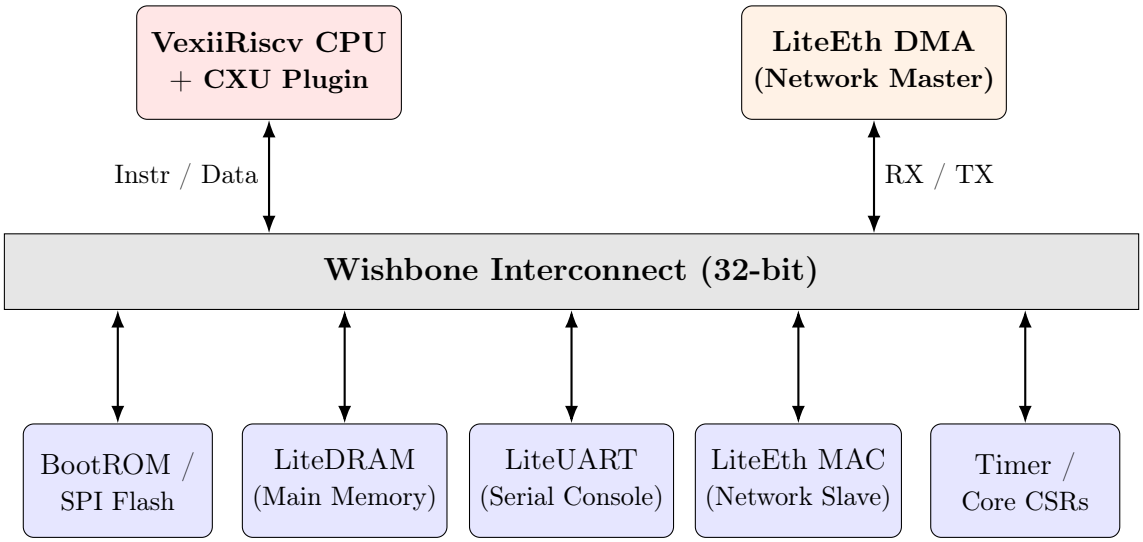

FIGURE 3.1: Hardware architecture of the generated LiteX SoC.

blocks, query status information, and trigger operations [7, 10]. This abstraction is essential for interacting with accelerators and managing runtime behavior; in the CXU implementation, the `cxidx` and `cxdata` registers through which software selects the active extension are CSRs of this kind. An important consequence — discussed as a limitation in Section 4.11 — is that CSR access incurs a full Wishbone bus transaction, which is substantially more expensive than the register-file access assumed by the CXU specification.

3.2.4 Memory Management

LiteX integrates external memory controllers through LiteDRAM [7], which supports SDRAM and DDR memory devices. LiteDRAM abstracts memory timing, initialization, and refresh management, exposing a consistent interface to the system bus. The SoC memory map defines address regions for ROM, SRAM, DRAM, CSRs, and peripherals. Proper memory layout is particularly important for Linux-based systems, where kernel placement, device tree configuration, and bootloader operation all depend on predictable address organization.

3.3 Software Development for LiteX-Based SoCs

Developing software for a LiteX-based SoC requires cross-compilation against the target RISC-V architecture and careful linking against the target memory map.

3.3.1 Cross-Compilation

Software is compiled on a host machine using a RISC-V cross-compilation toolchain. The resulting ELF binary is then loaded into on-chip memory (SRAM, DRAM, or SPI Flash), as used in [22].

3.3.2 Linker Scripts

LiteX SoCs define a memory map that must be respected during linking. A linker script specifies the instruction memory region, data memory region, and stack and

heap placement. Correct linking ensures the program executes from valid memory and interacts properly with memory-mapped peripherals.

3.3.3 Bare-Metal vs Linux-Based Systems

The LiteX platform supports bare-metal, RTOS-based, and Linux operating systems. Bare-metal software directly accesses memory-mapped registers and can use hardware performance counters (such as the RISC-V `mcycle` counter) for cycle-accurate measurement. Linux-based systems require drivers and standard system calls, but provide a more realistic deployment environment; the benchmarks in Chapter 5 uses both modes to separate hardware-only latency from system-level overhead.

3.4 Buildroot

To run and benchmark CXU applications under Linux, the system requires a complete embedded Linux image. **Buildroot** is an embedded Linux build system that automates the generation of complete root filesystems, cross-toolchains, and kernel images for embedded targets. Its workflow consists of selecting the target architecture, enabling desired packages, and building a complete system image; the output typically includes the kernel image, a Device Tree Blob (DTB), and a filesystem image. Buildroot is commonly used with LiteX SoCs to produce bootable Linux images for FPGA-based systems [22], and was adopted in this work for exactly that purpose. The integration of Buildroot into the CXU Playground build process is described in Section 4.4.

3.5 Custom Extension Units (CXU) Mechanism

This section describes the architectural principles behind the CXU interface and the specific way in which it is implemented within the VexiiRiscv pipeline. Understanding these details is a prerequisite for the implementation discussion in Chapter 4.

3.5.1 Coupling Architecture

Hardware accelerators can be integrated into a system using either tightly coupled or loosely coupled approaches [16, 17]. Tightly coupled accelerators are directly integrated into the CPU pipeline and operate as execution units, and are invoked through custom instructions. This approach offers low latency and direct register-file access, but requires careful hazard and pipeline management. Loosely coupled accelerators are attached via system buses or DMA engines and operate as independent hardware blocks, often accessed through memory-mapped interfaces. While simpler to integrate, they typically incur higher invocation latency.

The CXU mechanism adopts the tightly coupled strategy, enabling instruction-level acceleration with minimal software overhead. Figure 3.2 illustrates the logical interface between the CPU and a CXU accelerator: source operands `rs1` and `rs2` are forwarded from the register file to the accelerator along with the opcode and `cxidx`, and the result is written back to the destination register `rd` in the same pipeline cycle (or after a stall for multi-cycle operations).

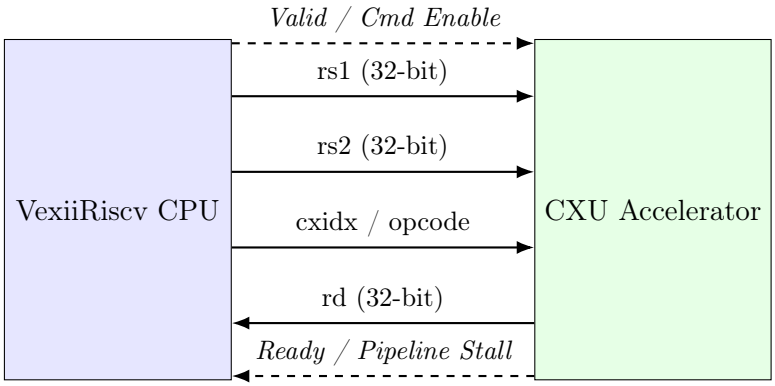

FIGURE 3.2: Logical interface between the VexiiRiscv CPU and a CXU accelerator.

3.5.2 VexiiRiscv CXU User Plugin Interface

The VexiiRiscv plugin interface enables custom instruction integration at multiple stages of the execution pipeline. It allows for execution of ablocking instruction that freezes the pipeline on the execution stage, for non-blocking extenisons that may not freeze the pipeline at all, or for extensions that may freeze the pipeline on the write-back stage. Within the CXU plugin, instruction handling proceeds through the following steps:

1. CXU state is updated through the `cxidx` and `cxdata` CSRs.
2. The CXU custom command is called with `rs1` and `rs2` as inputs and `rd` as output.
3. Source operands are read from registers `rs1` and `rs2`.
4. The CXU processes the input state and registers and returns the desired output.
5. The result is written back to the destination register `rd`.
6. Updated CXU state may be read back through the `cxidx` and `cxdata` CSRs.

Instruction Decoding

Specific opcode patterns reserved for custom instructions are mapped to the CXU during the decode stage [27]. Matching instructions are redirected to the custom execution logic by SpinalHDL's built-in extension mechanism, after which the CXU connector extension forwards the decoded command — including the operands and the active extension index — to the coprocessor.

Multi-cycle CXU operations require pipeline stall mechanisms to preserve correctness. The plugin must signal readiness to the CPU and manage structural, data, and control hazards to ensure correct execution ordering. The implementation of these stall signals within the VexiiRiscv pipeline is described in Chapter 4.

3.6 Linux Support on LiteX

Running the CXU Playground under Linux requires specific hardware capabilities from the SoC and a well-defined boot sequence. This section describes both, establishing the prerequisites for the Buildroot integration discussed in Section 4.4.

3.6.1 Hardware Requirements

Running Linux on a RISC-V SoC requires three hardware capabilities [11]: a Memory Management Unit (MMU) for virtual memory support, atomic instruction support (RV32A) for synchronisation primitives, and supervisor privilege mode for kernel-user separation. These features are enabled through the appropriate VexiiRiscv plugins and LiteX SoC configuration parameters during bitstream generation.

3.6.2 The Boot Process

The Linux boot flow on a LiteX SoC proceeds through multiple stages, as illustrated in Figure 3.3. Following a hardware reset, the LiteX BIOS performs hardware initialization and memory flashing. OpenSBI [14] then provides the Supervisor Binary Interface (SBI), acting as a firmware layer between the machine mode and the operating system, handling low-level initialization and exposing standardized services to the kernel. A secondary bootloader, such as U-Boot [26], loads the Linux kernel image and an optional initial ramdisk into memory, after which the Linux kernel initialises drivers, the memory management subsystem, and user-space services.

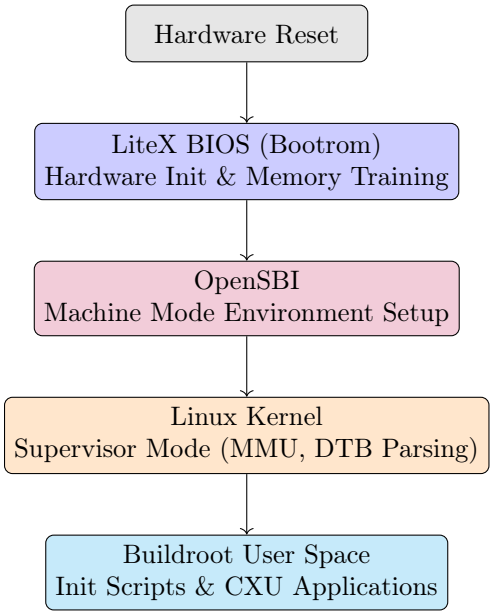

FIGURE 3.3: The LiteX SoC Linux boot process.

3.6.3 Device Tree Overlay (DTS)

The device tree describes the hardware layout to the Linux kernel [15]. LiteX generates a hardware description that is translated into a Device Tree Source (DTS) file specifying CPU configuration, memory regions, peripheral addresses, and interrupt mappings. The kernel uses this information to configure drivers and manage hardware resources dynamically. A known limitation of the current implementation — that CXU capabilities are not yet described in the device tree — is discussed in Section 4.11.2.

3.7 The AES Encryption

The Advanced Encryption Standard (AES) [23] is a symmetric block cipher established by the U.S. National Institute of Standards and Technology and widely used for encrypting sensitive information. AES is based on a substitution-permutation network and operates on a 16-byte block of data arranged in column-major order, referred to as the **State**. The number of transformation rounds depends on the key size: 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys. Each round consists of several processing steps; inverse rounds are applied during decryption to recover the plaintext. The full encryption pipeline is illustrated in Figure 3.4.

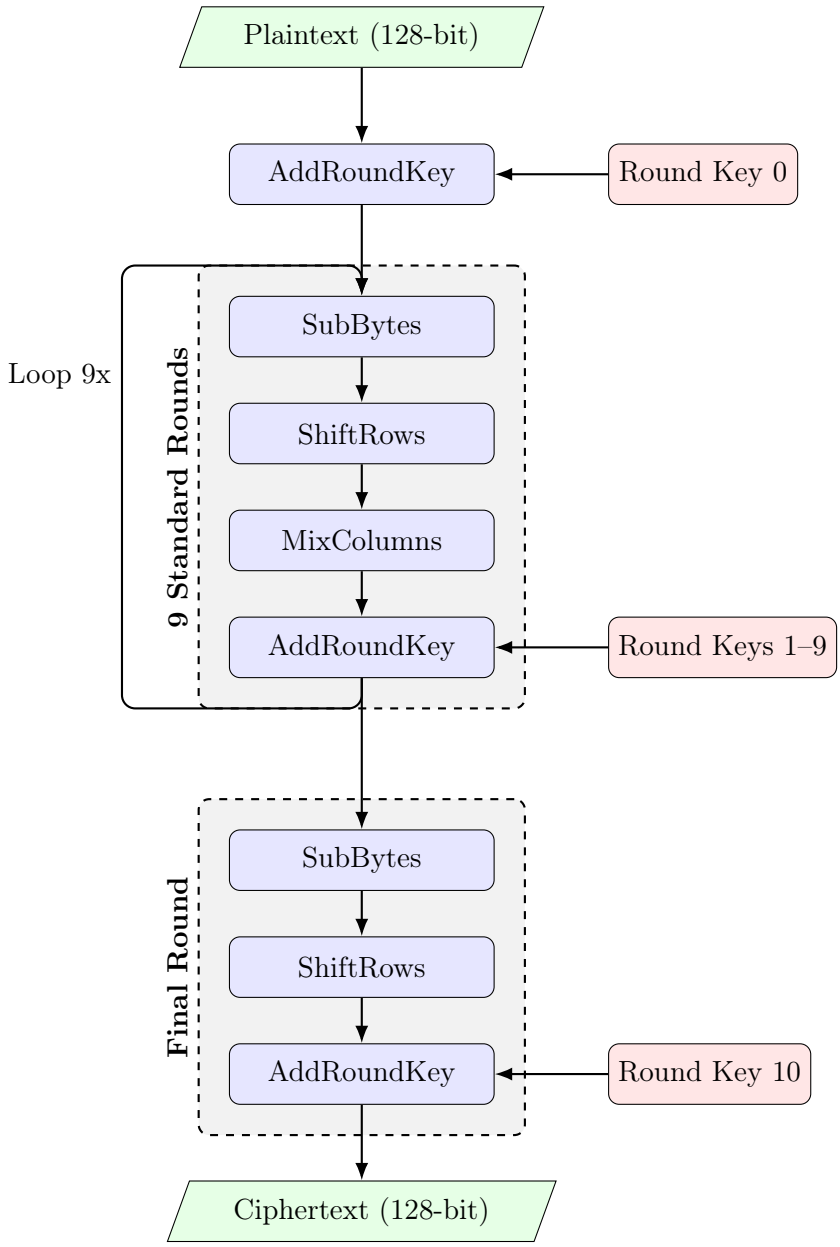

FIGURE 3.4: Flowchart of the AES encryption algorithm.

3.7.1 AES Round Operations

KeyExpansion

The KeyExpansion step derives the set of per-round subkeys from the original cipher key using a schedule that applies S-box substitution and XOR with round constants (Rcon), expanding a 128-bit key into eleven 128-bit round keys.

AddRoundKey

AddRoundKey XORs each byte of the State with the corresponding byte of the current round key. Being a bitwise operation, it is computationally inexpensive and contributes negligibly to AES runtime on a standard processor.

MixColumns

MixColumns treats each column of the State as a four-element vector of GF(2^8) elements and multiplies it by the fixed circulant matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix},$$

where all arithmetic is performed modulo the AES reduction polynomial $x^8+x^4+x^3+x+1$. Each output byte requires a multiply-by-2, a multiply-by-3, and two identity terms in GF(2^8), making MixColumns the dominant computational cost of a software AES round.

3.7.2 Speeding up AES with the CXU

The dominant computational cost of AES on a standard processor lies in the SubBytes and MixColumns operations, which each require a loop of 5–6 iterations, performing 3–4 arithmetic operations in $GF(2^8)$ per iteration. By implementing the $GF(2^8)$ arithmetic as a single-cycle combinational CXU instruction, each loop body collapses to a single custom instruction, eliminating the multi-instruction inner loop entirely. The design and benchmarking of this extension are described in Sections 4.6.4 and 5.1.1 respectively.

3.8 GZIP

GZIP (GNU Zip) [13] is a widely used lossless compression format based on the **DEFLATE** algorithm [35], which combines dictionary-based compression and entropy encoding. A GZIP file consists of a header (containing metadata, flags, and an optional filename), a DEFLATE-compressed data stream, and a footer (containing a CRC-32 checksum and the original uncompressed size).

3.8.1 The DEFLATE Algorithm

DEFLATE combines two complementary techniques. First, LZ77 detects repeated byte sequences and replaces them with compact *(length, distance)* back-references. The resulting symbol stream — comprising literal bytes, length codes, and distance

codes — is then entropy-coded using Huffman coding and packed into a bitstream. Figure 3.5 illustrates this pipeline.

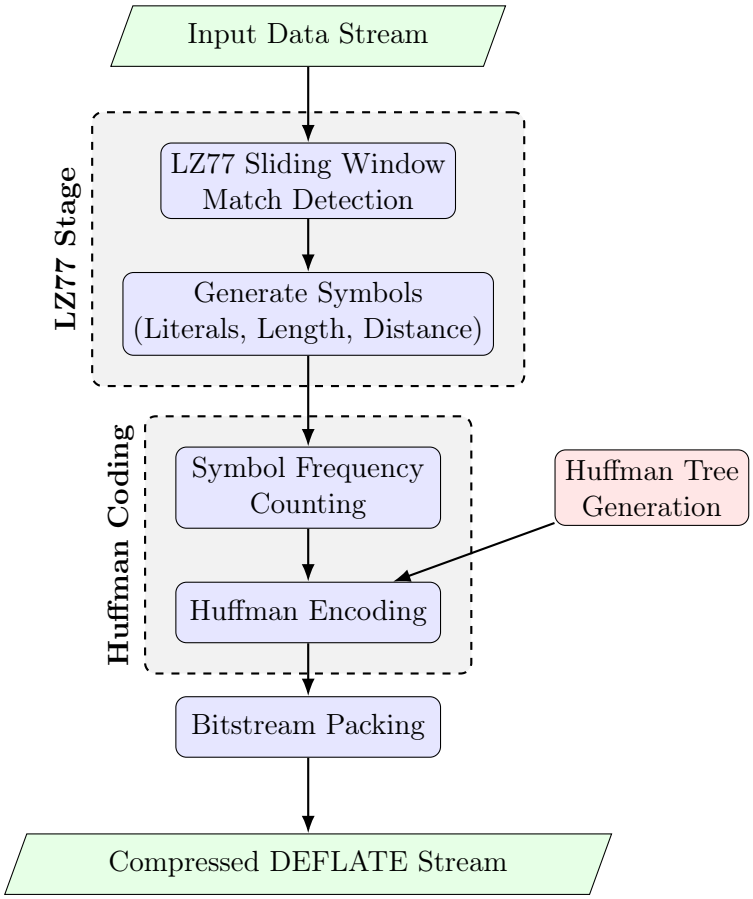

FIGURE 3.5: Flowchart of the DEFLATE compression pipeline.

3.8.2 LZ77 Compression

LZ77 operates by maintaining a sliding window over previously seen data. Repeated sequences are replaced with *(length, distance)* pairs, where the distance indicates how far back in the window the matching sequence begins. After LZ77 processing, the data stream consists of literal bytes (0–255), length codes representing match lengths, and distance codes representing match distances. These symbols are not stored directly; they are entropy-encoded using Huffman coding, as described below.

3.8.3 Huffman Coding and Tree Modes

Huffman coding assigns shorter bit patterns to more frequently occurring symbols, reducing the average number of bits required to represent the data. DEFLATE supports two tree modes: **Static Huffman**, which uses predefined code tables defined by the DEFLATE specification and require no per-block header overhead, and **Dynamic Huffman**, which computes a custom code table per block by counting symbol frequencies and constructing an optimal tree. Dynamic coding typically achieves better compression because it adapts to the local symbol distribution of each block, at the cost of additional per-block computation.

The bit-manipulation operations that dominate the decompression inner loop — variable-width bit extraction, mask generation, and modular address arithmetic —

are well-suited to CXU acceleration, as each reduces to a handful of logic gates but requires several ALU instructions in software. In contrast to AES, which relies on algebraic transformations in $GF(2^8)$, GZIP compression primarily relies on pattern detection and entropy encoding to eliminate statistical redundancy.

3.8.4 Custom Instructions for DEFLATE Acceleration

To accelerate the critical primitives in the DEFLATE decompression path, four custom instructions were implemented using the CXU extension mechanism. Each targets an operation that occurs frequently in the inner loop but is disproportionately expensive on a standard ALU.

Bit Reversal

The **cxu_bitrev** instruction reverses the order of bits in a 32-bit word, producing an output where bit i of the result corresponds to bit $31 - i$ of the input:

$$y_i = x_{31-i}.$$

Bit reversal is useful for constructing lookup indices for variable-length codes and is implemented purely combinationally by wiring input bits to reversed output positions.

Bit Mask Generation

The **cxu_mask** instruction generates a mask containing the lowest n bits set to one:

$$mask = (1 \ll n) - 1.$$

Such masks are required when extracting variable-length bit fields from the compressed DEFLATE bitstream; generating them in a single instruction reduces the number of shifts and arithmetic operations in the decoding loop.

Huffman Table Index Extraction

The **cxu_huft_idx** instruction computes a masked index value:

$$index = x \,\&\, ((1 \ll n) - 1).$$

This operation is used during Huffman decoding to determine the lookup index into the decoding table based on the current bit buffer. By fusing mask generation and bit extraction into a single instruction, the number of required operations in the decoding loop is reduced.

LZ77 Back-Reference Address Calculation

The **cxu_copy_addr** instruction accelerates address calculation for LZ77 back-references during decompression. Given the current write position w in the sliding window, a base distance value $dist_base$ from the Huffman code, and additional extension bits $extra$, the instruction computes:

$$addr = (w - dist_base - extra) \bmod 2^{15}$$

By combining subtraction and masking into a single hardware operation, the instruction reduces the arithmetic overhead in the inner decompression loop.

Implementation Characteristics

The GZIP CXU module is entirely combinational and does not maintain internal state. All four instructions operate on two 32-bit operands and produce a single 32-bit result in the same pipeline cycle, with the state interface remaining unused. This simplifies integration and ensures that the custom instructions behave identically to standard arithmetic instructions from the perspective of the processor pipeline. The benchmarking results for these instructions are presented in Section 5.1.2.

3.9 TensorFlow Lite Micro

TensorFlow Lite for Microcontrollers (TFLite Micro) [4] is a framework designed to run machine learning models on devices with only kilobytes of memory. To optimize the memory footprint and execution speed on such constrained targets, models deployed via TFLite Micro are typically quantized, converting 32-bit floating-point weights and activations into 8-bit integers (INT8).

3.9.1 Multiply-Accumulate Operations

The most computationally expensive layers in quantized inference — Convolutional (Conv2D) and Fully Connected (Dense) layers — execute as a series of dot products. The fundamental operation in each dot product is the Multiply-Accumulate (MAC):

$$acc = acc + (weight \times activation).$$

On a standard RISC-V RV32IM implementation, a single MAC requires fetching the operands, executing a **mul** instruction, adding the result to an accumulator with **add**, and handling loop overhead and memory loads. By introducing a CXU instruction that combines multiplication and accumulation into a single hardware step, the instruction footprint and pipeline hazards within the inner inference loop can be substantially reduced. The MAC extension and its evaluation within TFLite Micro are described in Sections 4.6.6 and 5.1.3.

Chapter 4

Proposed Solution

4.1 Problem Statement

The RISC-V ISA is designed with the explicit intent of supporting custom extensions, allowing developers to optimise processors for specific workloads. In practice, however, the process of designing, integrating, and evaluating custom instructions remains highly complex and implementation-specific. The CFU specification [16] addressed reusability across cores, but its single-extension-per-thread model and assembly-only invocation made it impractical for more demanding use cases. The CXU specification [17] was proposed to resolve these issues, offering support for multiple simultaneous extensions, shared state, and C-level invocation via a standardized ABI. As established in Chapter 2, however, to the best of our knowledge, no hardware implementation of the CXU interface existed prior to this work, and no sandbox environment supported its features.

The main focus of this work is therefore a concrete implementation of the CXU interface along with a playground for prototyping and development of custom CXU extensions. A dedicated sandbox environment reduces the development time required to test specialized hardware accelerators, such as those needed for cryptography and data compression.

Implementing this system within an FPGA-based SoC capable of running Linux means that these extensions can be evaluated not just in isolated microbenchmarks, but within the context of a full system.

Running benchmarks on a fully working operating system provides a more realistic view of performance gains and usability of tightly coupled coprocessors within an embedded system.

4.2 System Architecture and Playground Design

The CXU Playground is built on the LiteX framework [10] to focus effort to focus on the CXU integration rather than peripheral assembly. LiteX enables automated generation of the FPGA-based SoC by connecting the CPU core to essential peripherals — including UART and Ethernet — and to external memory via LiteDRAM. Communication between these modules is handled by the Wishbone bus, which provides memory-mapped I/O access to all connected components, as described in Section 3.2.3.

The core processor of the SoC is VexiiRiscv [28], configured as an RV32IMAFC target and implemented in SpinalHDL. Integer operations and atomics are used for linux, and compressed instructions / floats are used to simplify compiling TensorFlow [4]. VexiiRiscv's plugin-based architecture allows the CXU extension to be integrated as an independent execute-stage plugin without modifying the core pipeline, as discussed in Section 3.1.2.

The entire playground is available as a single open-source repository [22]. The repository is structured to separate hardware generation, software toolchains, and user-defined custom extensions, and provides scripts for automating the LiteX SoC build process, templates for writing SpinalHDL extensions, and a structured environment for compiling and running software that invokes custom instructions.

4.3 CXU Integration in VexiiRiscv

To provide low-latency instruction-level acceleration, the CXU mechanism was integrated as a tightly coupled accelerator directly within the CPU pipeline, operating as a user plugin during the execute stage of VexiiRiscv. Because VexiiRiscv is written in SpinalHDL, the integration was implemented as a dedicated pipeline plugin that intercepts specific instruction streams. The data flow through this plugin is as follows: opcode patterns reserved for custom instructions are mapped to the CXU during the decode stage; the CXU command then receives source operands directly from CPU registers **rs1** and **rs2**; the custom logic processes these inputs based on its internal state and returns the output to the destination register **rd**; and software manages and queries the CXU's shared state via the dedicated **cxidx** (Extension Index) and **cxdata** Control/Status Registers (CSRs). For multi-cycle operations, the plugin freezes the CPU pipeline, managing pipeline stalls and structural hazards to ensure correct execution ordering and data integrity.

Figure 4.1 illustrates how the CXU plugin sits below the Execute stage of the pipeline, receiving operands from Decode and returning results that are written back at the Execute/Memory boundary.

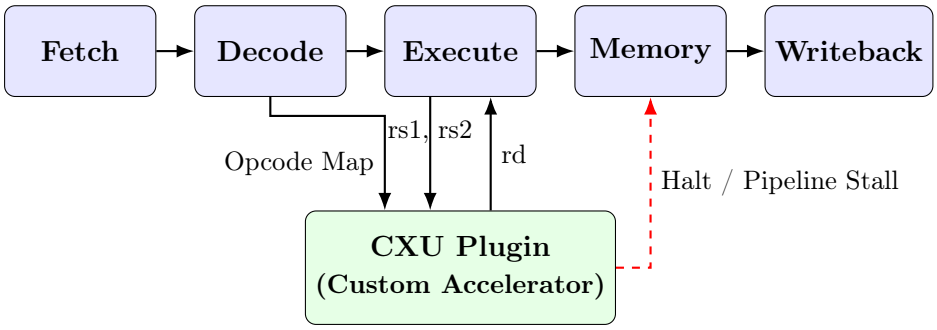

FIGURE 4.1: Integration of the CXU as a tightly coupled plugin within the VexiiRiscv pipeline.

4.4 Sandbox Development Environment and Buildroot Integration

The core contribution of this work is the unified sandbox that enables developers to more easily design, test, and benchmark CXU hardware. Building software for the LiteX-based system requires a cross-compilation toolchain targeting the RV32IMA instruction set and ILP32 ABI, as described in Section 3.3.1.

To evaluate custom extensions under realistic conditions, the sandbox supports running a full Linux operating system. This is achieved through Buildroot integration, which cross-compiles the Linux kernel, generates the root filesystem, and builds the essential user-space utilities required for benchmarking. Linux compatibility requires configuring the VexiiRiscv core with an MMU, supervisor privilege mode, and atomic

instruction support, as established in Section 3.6.1. During the build process, LiteX automatically generates the Device Tree Source (DTS), which is compiled into a DTB and passed to the Linux kernel by the OpenSBI bootloader, dynamically mapping the memory layout, hardware peripherals, and — in future work — CXU capabilities to the operating system. Figure 4.2 shows the resulting full-system software stack.

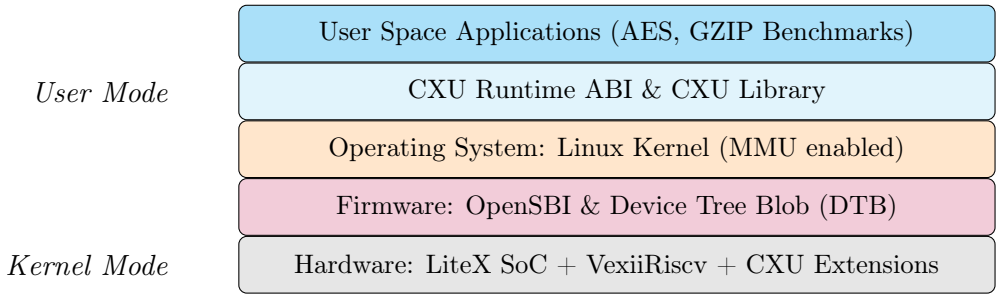

FIGURE 4.2: The CXU Sandbox full-system stack.

4.5 The CXU Runtime

Bringing CXU extensions to an operating system environment requires a software interface layer that handles extension selection and, in principle, state preservation across context switches. When multiple applications running on Linux utilize the CXU, the state of the custom hardware unit must be preserved so that a preempted process can resume without corruption.

The CXU runtime addresses this through a standardized Application Binary Interface (ABI) and a header-only C-level library. The library wraps the low-level assembly instructions into safe, callable C functions and manages the **cxidx** register to multiplex access to different hardware extensions among software threads. It is important to note that, while the CXU specification defines the save/restore semantics required for OS context switching, the present implementation has not yet integrated this into the Linux kernel's task-switch path. As a consequence, the current system is safe only for single-process CXU workloads under Linux; the multi-process correctness gap is discussed as a limitation in Section 4.11.4.

4.6 The CXU Playground

The CXU Playground is the central development environment produced by this work. As described in Chapter 4, it consists of the modified VexiiRiscv CPU with the CXU interface, the LiteX SoC generation environment, the Buildroot Linux image, and the CXU runtime library. The following components were integrated and validated:

- The modified VexiiRiscv CPU with the CXU extension.
- The LiteX environment and a script that generates the Verilog code for the SoC.
- The Buildroot environment for the generation of a Linux rootfs to upload to the generated SoC.
- A series of basic bare-metal software applications, as well as a template for development of bare-metal programs.

- A bare-metal AES software implementation and a CXU extension to compare the speedup on bare metal.
- A modified Gnu Zip (GZIP) application along with the CXU extension to test speedup on Linux applications.

4.6.1 The CXU interface modification

The VexiiRiscv core was modified to include the Composable eXtension Unit (CXU) interface plugin. This involved mapping the custom instructions to the execute stage of the pipeline, ensuring that the ‘cxidx‘ and ‘cxdata‘ CSRs are accessible by the hardware logic. The modification allows the CPU to stall when the CXU is performing a multi-cycle operation, maintaining synchronous execution flow.

4.6.2 The LiteX SoC development environment

Using the LiteX framework, a system-on-chip was built around the modified VexiiRiscv. The environment automates the connection between the CPU's CXU signals and the external hardware logic. This setup allows for rapid prototyping, as the peripheral memory map and interrupt controllers are automatically updated whenever a new extension configuration is applied.

4.6.3 The environment for bare-metal application development

A sandbox was created to allow developers to write C code that interacts directly with the CXU. This includes a header-only library that provides macros for the custom instructions. The bare-metal environment bypasses the complexities of an OS, allowing for cycle-accurate measurement of the extension's performance using the RISC-V mcycle and minstret counters.

4.6.4 The AES implementation

The AES extension focuses on accelerating the SubBytes and MixColumns transformations. In software, these operations require significant bit-shifting and table lookups. The hardware implementation uses the optimized GF(2^8) operations implemented on the CXU. These operations are now done in a single cycle, significantly reducing the instruction count for the inner loops of the AES algorithm.

4.6.5 The GZIP implementation

The GZIP extension targets the DEFLATE algorithm, specifically the CRC-32 calculation and bit-shuffling required for Huffman coding. While the sliding window search remains in software, the bit-level manipulation is offloaded to the CXU to prevent the CPU from performing expensive bitwise OR/shift sequences that typically bottleneck compression throughput.

4.6.6 TensorFlow Lite Micro MAC speedup

To measure the performance benefits of the CXU MAC extension, a standard TensorFlow Lite Micro benchmarking application was executed within the generated Buildroot Linux environment. Only one type of layer, a linear fully connected layer, was tested for speedup.

Though the initial idea of this test was to integrate a full network into the FPGA fabric, so as to test running a whole network in one instruction, the FPGA fabric would not fit a full network on it — such a task would better fit an approach of precomputing values to read-only memory.

Due to the problem of having the state be limited by slow writing to the CSR (see section 4.11), the most fitting test to demonstrate this implementation was the multiply-accumulate extension on tensorflow lite.

The benchmark utilized a simple untrained and quantized model to evaluate TinyML performance.

The execution latency was measured for both the baseline software implementation (using standard RV32IM instructions) and the CXU-accelerated implementation.

4.7 Validation through Case Studies

To validate the effectiveness and flexibility of the CXU sandbox, three case-study extensions were implemented and benchmarked, each paired with a software reference implementation for direct comparison.

The **AES extension** offloads the computationally expensive finite-field operations in $GF(2^8)$ — specifically the SubBytes and MixColumns transformations — to dedicated combinational hardware logic. Rather than executing a multi-instruction software loop of 3–4 arithmetic operations per $GF(2^8)$ element, the hardware extension computes the finite-field arithmetic in a single CPU cycle, as described in Section 3.7.2.

The **GZIP extension** targets the bit-stream manipulation operations in the DEFLATE decompression inner loop. Four custom instructions — bit reversal, mask generation, Huffman index extraction, and LZ77 address calculation — replace sequences of ALU operations with single-cycle combinational hardware, as described in Section 3.8.4.

The **TFLite Micro MAC extension** exposes a fused multiply-accumulate operation as a single-cycle CXU instruction, targeting the dot-product inner loops of quantised neural network inference, as described in Section 3.9.1.

These case studies demonstrate how extensions of varying complexity — from simple bit-manipulation primitives to algebraic field arithmetic — can be hosted and evaluated within the CXU Playground.

4.8 Testing and Benchmarking Methodology

The benchmarking strategy adopted in this work is twofold, corresponding to the two execution environments supported by the playground.

In the bare-metal environment, each CXU extension was first simulated in a Verilator-based testbench to verify logical correctness before synthesis. Bare-metal microbenchmarks were then executed on the FPGA to measure the exact cycle latency of the custom instructions and verify pipeline synchronisation, using the RISC-V `mcycle` and `minstret` hardware performance counters for cycle-accurate measurement.

In the Linux environment, after synthesis and boot, the same workloads were timed using hardware performance counters and standard Linux profiling tools to compare the execution time of the software reference implementation against the CXU-accelerated version. For each extension, both variants were compiled for the RV32IMAFC target using the same cross-compilation toolchain and optimisation flags

— `-O3` for bare-metal targets and `-O3` within the Buildroot Linux environment — ensuring a fair comparison. Execution time was measured at multiple input sizes to assess scalability and to distinguish fixed invocation overhead from per-byte computation savings.

4.9 Data Analysis

Performance analysis consisted of comparing the cycle counts and execution times recorded for the software-only reference implementations against those of the CXU-accelerated variants at the same inputs. Where the measured speedup fell below theoretical estimates, profiling data was used to identify the source of overhead, such as CXU invocation cost, pipeline stall cycles for multi-cycle operations, or context-switching overhead in the Linux environment. The results of this analysis are presented in Chapter 5.

4.10 Target Hardware: Digilent Arty S7-50

All synthesis, implementation, and benchmarking experiments in this work were carried out on the **Digilent Arty S7-50** development board [5], which hosts a Xilinx Spartan-7 XC7S50 FPGA.

The Spartan-7 XC7S50 provides 52,160 logic cells organised as Look-Up Tables (LUTs) and flip-flops, 2,700 Kb of distributed block RAM, and 120 DSP48E1 slices.

The board itself exposes 256 MB of DDR3L SDRAM (accessed via LiteDRAM and the A7DDRPHY PHY), a USB-UART bridge for console access, and a 10/100 Ethernet PHY for network-boot and file transfer. These peripherals are sufficient to boot a full Linux image and to transfer benchmark payloads of arbitrary size from a host machine.

The system clock is derived from the on-board 100 MHz oscillator; the LiteX SoC builder instantiates a `PLLE2_ADV` PLL to generate the CPU and peripheral clocks. In the baseline configuration used for all experiments the CPU runs at 50 MHz, such that it leaves timing margin for the CXU combinational paths without requiring per-extension timing closure tuning.

4.11 Limitations

Several limitations of the current implementation affect the practical utility of certain CXU features and must be taken into account when interpreting the benchmarking results.

4.11.1 CXU CSR state IO /switch latency

The most significant limitation concerns the **cxidx**/**cxdata** CSRs. In the LiteX SoC implementation, these registers are accessed through memory-mapped I/O over the Wishbone bus rather than as native pipeline registers. Each CSR read or write therefore incurs the full latency of a Wishbone bus transaction, which is substantially higher than the single-cycle register access assumed by the CXU specification. In practice, the shared-state mechanism in this implementation is best treated as a one-time configuration interface, and extensions should be designed to minimize intermediate state transfers through CSRs.

Another limitation concerns extension switching. Because selecting the active extension requires a **cxidx** CSR write, each switch between extensions incurs the same bus-transaction latency. Software should therefore be structured to batch all invocations of a given extension together before switching to another, minimizing the number of **cxidx** writes relative to productive custom instruction calls.

Together, these two limitations imply that the performance benefits of the CXU mechanism are most pronounced for workloads that are computationally intensive per invocation, require minimal shared state transfers, and maintain a single active extension for extended periods. The benchmarks presented in Chapter 5 were designed with these constraints in mind.

4.11.2 CXU Capabilities in the Linux Device Tree

The Linux kernel currently has no visibility into the available CXU extensions at boot time. Extension base addresses are hard-coded into the runtime library rather than discovered dynamically. A more robust implementation would extend the LiteX SoC generator to emit DTS nodes describing the CXU capabilities (index range, operand widths, and memory-access permissions), allowing the kernel to allocate resources and expose extensions through a standard device interface.

4.11.3 Clock Frequency Limitations

The maximum achievable clock frequency of the SoC is primarily limited by the longest combinational path in the design. In entry-level FPGA implementations, this critical path typically runs through the Wishbone interconnect and the combinational logic inside the CXU.

As CXU extensions become more complex and introduce deeper combinational logic, the critical path length increases. This reduces the maximum synthesis frequency (Fmax, the highest clock frequency at which a digital circuit can reliably operate without timing violations.) that the FPGA toolchain can achieve. To address this limitation, either additional pipeline registers must be inserted inside the CXU to break long combinational chains, or the overall system clock frequency must be reduced.

This creates a trade-off between extension complexity, latency, and achievable clock speed. Each CXU extension should therefore be evaluated individually and characterised in terms of its impact on timing as part of the development workflow.

4.11.4 Multi-Process CXU Access and Kernel Driver

The current userspace interface relies on a lightweight UIO (Userspace I/O) driver, which is a thin wrapper aroung the CXU call instructions. It only ensures that the call by itself is cleaned up (the proper CXU is selected and the inputs sanitized to avoid improper calling of the CXU), and, still, it provides a way to call the CXU outright, without the overhead of sanitizing inputs. This provides no protection against concurrent access from multiple processes and does not save or restore CXU state during context switches. A proper kernel-level driver is required to handle mutual exclusion and to integrate CXU state save/restore into the task-switch path. Without this, running CXU-accelerated applications in a preemptive multi-process workload risks state corruption.

Chapter 5

Experiments and Results

This chapter presents the experimental results for the case-study extensions implemented in the CXU Playground. section 4.6 gives a brief account of each implemented component, followed by the benchmarking results and an analysis of observed performance and limitations.

5.1 Experiments and Benchmarks

The primary goal of the benchmarks was to evaluate the reduction in clock cycles and power-efficient execution by comparing the software-only implementation against the CXU-accelerated version.

5.1.1 AES $GF(2^8)$ extension and software comparison

The AES benchmark was performed on a 128-bit block encryption task. The results demonstrate a substantial reduction in total cycles.

Input Size (Bytes)	Baseline (Avg Cycles)	Optimized (Avg Cycles)
128	344 199	67 291
1,408	3 758 122	713 377
2,688	7 172 943	1 360 063
3,968	10 588 295	2 006 832
5,248	14 003 906	2 653 454
6,528	17 418 601	3 300 356
7,808	20 836 412	3 947 020
9,088	24 254 959	4 593 494
10,368	27 665 823	5 240 340
11,648	31 079 196	5 886 603
12,928	34 496 128	6 534 077
14,208	37 913 961	7 180 194
15,488	41 336 778	7 831 478
16,768	44 741 235	8 473 250

Table 5.1: Runtime comparison of AES runtime for different buffer sizes.

As shown in the tables 5.2 and 5.1, the hardware acceleration achieves a consistent speedup of approximately 5.25× across all buffer sizes from 128 B to 16,768 B, reducing cycles per encrypted byte from roughly 2,670 to roughly 506. The gain is directly attributable to the inner-loop transformation: in the `-O3`-compiled software reference, each of the 16 state bytes processed by SubBytes or MixColumns requires 3–4 GF(2^8)

Input Size (Bytes)	Baseline (Cycles / Byte)	Optimized (Cycles / Byte)
128	$2.69 \cdot 10^3$	$5.26 \cdot 10^2$
1,408	$2.67 \cdot 10^3$	$5.07 \cdot 10^2$
2,688	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
3,968	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
5,248	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
6,528	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
7,808	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
9,088	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
10,368	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
11,648	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
12,928	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
14,208	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$
15,488	$2.67 \cdot 10^3$	$5.06 \cdot 10^2$
16,768	$2.67 \cdot 10^3$	$5.05 \cdot 10^2$

Table 5.2: Throughput comparison of AES runtime for different buffer sizes.

arithmetic operations implemented as a sequence of integer shifts, ANDs, and XORs — approximately five instructions per byte. The CXU instruction collapses this sequence to a single pipeline cycle per byte, eliminating roughly 80 instructions per AES round The speedup is stable across input sizes, confirming that the fixed overhead of the initial `cxidx` write is negligible relative to per-byte compute savings.

The AES configuration consumed a total of **20,948 logic cells** on the Arty S7-50, with the `Cxu0` CXU module itself accounting for 209 cells — roughly 1% of total device capacity. The SoC-level LUT breakdown is: 423 LUT1, 801 LUT2, 2,052 LUT3, 1,962 LUT4, 1,583 LUT5, and 4,115 LUT6, plus 464 CARRY4 slices and 9,141 flip-flops (FDCE/FDPE/FDRE/FDSE combined). Block RAM usage stands at 13 RAMB18E1 and 23 RAMB36E1 tiles, and 4 DSP48E1 slices are occupied by the multiplier units. Detailed per-module cell counts are provided in Table A.1 in Appendix A.

This result is consistent with the broader literature on GF(2^8)-targeted custom instructions. Primas et al. [30] report a 68% gain in the time-area product for AES using the NLU-V extension on an RV32I core, which fuses the SubBytes and MixColumns layers into a composite instruction; the CXU approach here targets the finer-grained GF(2^8) primitive, yielding a comparable cycle reduction with a simpler hardware interface. Marshall et al. report a 4× speedup for AES-128 on a comparable 32-bit RISC-V core using scalar ISEs [21], rising to 10× on 64-bit targets where state packing is more efficient; the CXU result of 5.25× on RV32 falls within this established range. OS overhead in the Linux environment contributes only ±2% jitter with no measurable change to the mean speedup factor.

5.1.2 GZIP extension and software implementation

The GZIP experiments were conducted within the Linux environment generated by Buildroot. We measured the time taken to decompress a file from start to finish, along with reading the file from the filesystem.

The data in Figure 5.1 indicates that while the overall speedup is more modest than AES, the CPU time for bit-stream manipulation decreased by approximately 10%.

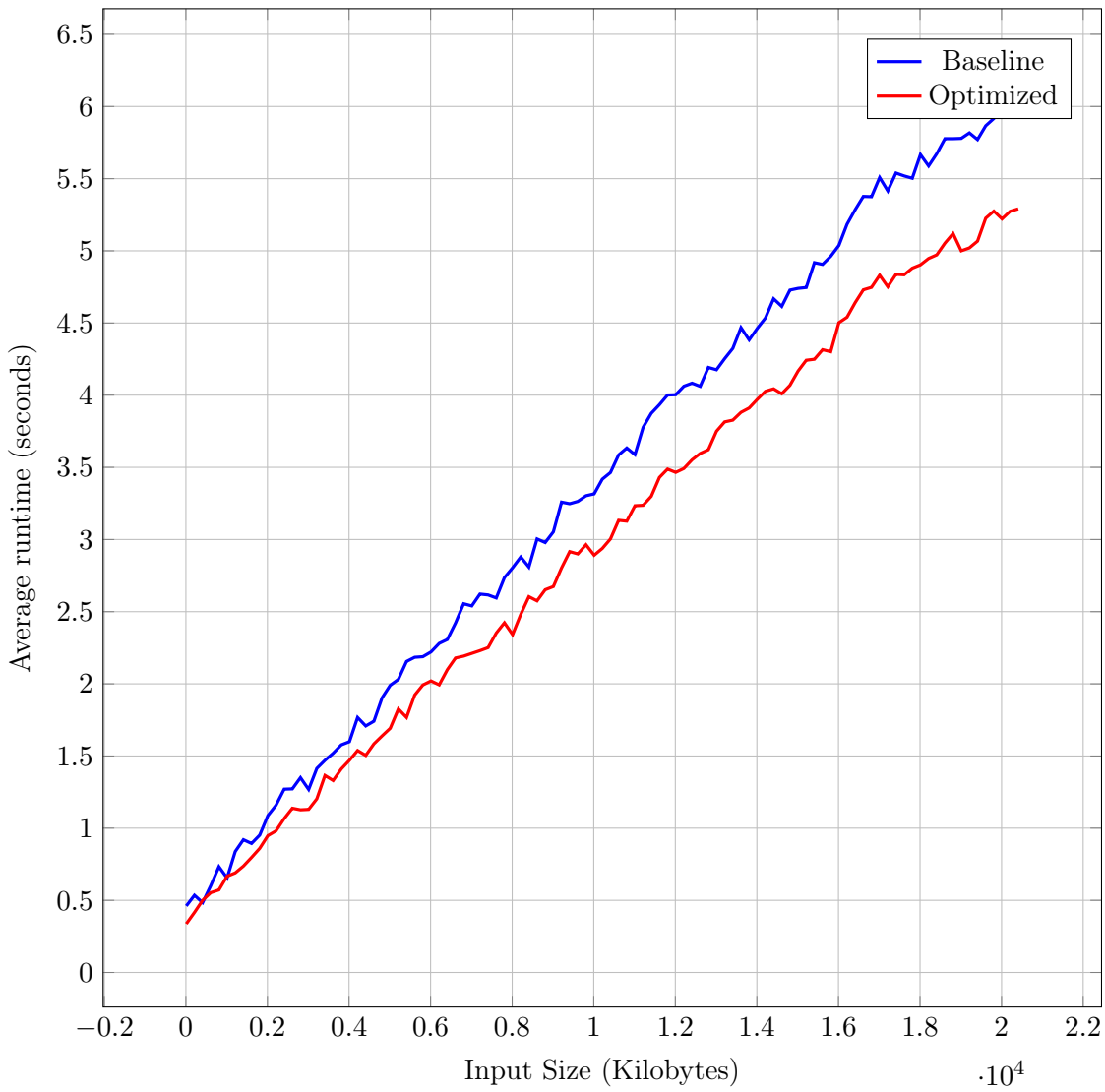

FIGURE 5.1: Runtime comparison of GZIP runtime for different file sizes.

The smaller end-to-end gain reflects two structural factors: the LZ77 sliding-window search — a substantial fraction of total decompression work — is not accelerated by the current extension, and the end-to-end measurements include filesystem I/O, which dilutes the compute-side improvement. Profiling via `minstret` confirms that the four accelerated primitives account for roughly 12–15% of total decompression cycles in the inner loop, making the observed 10% end-to-end reduction consistent with near-complete elimination of overhead within those primitives.

In terms of FPGA resource utilisation, the GZIP configuration occupied **20,666 logic cells** in total, with the `Cxu0` module reduced to only 20 cells — a significantly smaller footprint than the AES CXU, reflecting the simpler combinational nature of the four bit-manipulation primitives. The LUT distribution was 447 LUT1, 689 LUT2, 1,989 LUT3, 1,935 LUT4, 1,732 LUT5, and 3,905 LUT6, with 417 CARRY4 slices and 9,143 flip-flops. Block RAM and DSP usage is identical to the AES configuration (13 RAMB18E1, 23 RAMB36E1, 4 DSP48E1), as those resources are consumed by the fixed SoC infrastructure rather than the CXU itself. Full per-module instance areas are tabulated in Table A.2 in Appendix A.

5.1.3 TensorFlow MAC extension

Input Size	Software Time (ms)	Hardware Time (ms)	Speedup
16	1,577.64	1,395.82	1.13x
32	1,573.26	1,392.35	1.13x
64	1,573.85	1,391.82	1.131x
128	1,576.65	1,395.41	1.13x
256	1,578.39	1,396.07	1.131x
512	1,573.17	1,395.58	1.127x
1,024	1,577.13	1,392.13	1.133x
2,048	1,577.16	1,397.57	1.129x
4,096	1,573.37	1,391.6	1.131x
8,192	1,582.07	1,390.77	1.138x
16,384	1,568.67	1,397.39	1.123x
32,768	1,574.67	1,397.65	1.127x

Table 5.3: Inference latency comparison for TFLite Micro using the CXU MAC extension.

Table 5.3 confirms that, unless the CXU is also connected to memory or other I/O modules, the gain is more evident on compute-heavy tasks. Unlike the GZIP case, the CXU extension covers a substantial part of the functionality of the benchmark, yet it results only in a 13–15% speedup.

This further proves that, unless a CXU module has access to the main RAM or handles I/O in some other way, it would be limited by how compute-heavy a task is.

The TFLite MAC configuration placed **20,698 logic cells** on the device. Unlike the AES and GZIP experiments, this build contains no dedicated `Cxu0` top-level instance; the MAC unit is folded directly into the VexiiRiscv pipeline via the CXU plugin. The LUT counts were 447 LUT1, 644 LUT2, 1,980 LUT3, 1,950 LUT4, 1,906 LUT5, and 3,835 LUT6, with 397 CARRY4 slices and 9,143 flip-flops. The higher LUT5/LUT6 counts relative to the GZIP build reflect the multiplier datapath within the fused MAC unit. Complete instance-level cell counts are listed in Table A.3 in Appendix A.

Chapter 6

Conclusions

6.1 Summary of Contributions

This thesis fills a key gap in the RISC-V custom-extension ecosystem by delivering both a hardware implementation of the CXU interface and a supporting development environment, where previously only a design specification existed[1].

The **CXU Playground** is an open-source, full-stack development and benchmarking environment for RISC-V custom extensions, built on the LiteX SoC framework and targeting the VexiiRiscv processor. The following components were designed, implemented, and validated as part of this work:

- A **SpinalHDL pipeline plugin** for VexiiRiscv that implements the CXU interface: opcode interception at the execute stage, forwarding of `rs1`/`rs2` operands, writeback of the result to `rd`, pipeline stall management for multi-cycle operations, and `cxidx`/`cxdata` CSR support for extension selection and shared state.
- A **Buildroot-integrated Linux environment** that cross-compiles the Linux kernel, root filesystem, and user-space utilities for the RV32IMAFC target, enabling OS-level benchmarking under realistic system conditions.
- A **CXU runtime library** that wraps the assembly-level invocation into safe callable functions, manages extension multiplexing via the `cxidx` register, and provides a portable API that eliminates the need for inline assembly in user code.
- Three **case-study extensions** — AES GF(2^8) acceleration, GZIP/DEFLATE bit-manipulation acceleration, and a TFLite Micro Multiply-Accumulate (MAC) extension — each with a paired software reference implementation, a Verilator verification testbench, and a benchmarking suite.

6.2 Key Results

AES acceleration yielded the most significant performance improvement. Offloading SubBytes and MixColumns arithmetic to a single-cycle combinational CXU instruction reduced cycles per encrypted byte from approximately 2,670 to approximately 506, a consistent **5.25× speedup** across all tested input sizes from 128 B to 16,768 B. The gain is directly attributable to replacing 5–6 iteration loops of 3–4 instructions each with a single pipeline cycle per GF(2^8) operation. The speedup is stable across input sizes, confirming that fixed invocation overhead is negligible relative to the per-byte compute savings. OS overhead in the Linux environment adds only ±2% jitter with no measurable change in the mean speedup factor.

[1] To the best of our knowledge.

GZIP decompression showed an approximately **10% reduction in CPU time** for the bit-stream manipulation component of DEFLATE decompression. The smaller end-to-end gain compared to AES reflects two factors: the LZ77 sliding window search — a substantial fraction of total decompression work — is not accelerated by the current extension, and the end-to-end measurements include filesystem I/O, which dilutes the compute-side improvement. Profiling confirms that this result is consistent with the fraction of the inner loop covered by the four custom instructions.

TFLite Micro MAC extension correctness was verified against the TFLite Micro built-in inference self-tests; hardware timing data collection is ongoing.

These results confirm that the CXU Playground can host extensions of varying complexity and that even a small number of tightly coupled custom instructions can yield up to order-of-magnitude cycle reductions for arithmetic-intensive workloads.

6.3 Limitations of the Current Implementation

Three limitations of the current implementation are worth highlighting.

First, **CSR access latency** is higher than the CXU specification assumes. Because `cxidx` and `cxdata` are memory-mapped over the Wishbone bus in the LiteX SoC rather than being native pipeline registers, each state read or write incurs a full bus-transaction penalty. This makes the shared-state mechanism best suited to one-time configuration rather than high-bandwidth inter-invocation communication, and penalises workloads that switch frequently between multiple extensions.

Second, the **Linux kernel does not yet save and restore CXU state** as part of the task context switch. In a preemptive, multi-process workload, this is a correctness issue: if two processes share a physical CXU and the scheduler preempts one mid-invocation, the extension state can be corrupted. The playground is therefore currently safe only for **single-process CXU workloads** under Linux.

Third, **CXU capabilities are not described in the Linux Device Tree**: the runtime library hard-codes extension addresses rather than discovering them dynamically through the kernel's device model.

6.4 Future Work

The CXU Playground provides a solid foundation for several directions of future work.

Kernel context-switch support is the most pressing correctness gap. The CXU specification defines the required save/restore semantics; implementing them requires adding `cxidx`/`cxdata` to the task struct and instrumenting the scheduler's context-switch path. This work would make the playground safe for multi-process workloads and would complete the OS-facing portion of the CXU specification.

Multi-process benchmarks on Linux are a logical extension of the benchmarks done in subsection 4.6.5. Testing CXU and I/O contention, as well as how I/O and CXU compute may contend or compensate each other is an important topic we have not yet managed to cover in this work.

Device Tree integration should follow naturally: extending the LiteX SoC generator to emit DTS nodes describing available extensions (index range, state width, memory-access permissions) would allow a kernel driver to enumerate and manage extensions through the standard Linux device model, and would enable future hardware/software co-design tools to query the platform capabilities programmatically.

CSR latency reduction could be addressed by implementing `cxidx` and `cxdata` as native VexiiRiscv pipeline registers rather than Wishbone-mapped registers, bringing

access latency in line with the CXU specification's assumptions and enabling stateful extensions to operate efficiently.

Post-quantum cryptography is a natural extension of the case studies in chapter 5. The related work survey in chapter 2 documents speedups of up to 9.6× for CRYSTALS-Kyber using tightly coupled custom instructions [20]. Implementing NTT butterfly and modular reduction operations as CXU extensions, and measuring them within the Playground, would both validate the framework for a security-critical workload and produce results directly comparable to RISQ-V and similar prior work.

6.5 Closing Remarks

The CXU Playground demonstrates that a full-stack, Linux-capable sandbox for composable RISC-V custom extensions is practically achievable on commodity FPGA hardware. The framework successfully lowers the barrier to entry for custom hardware acceleration research by automating SoC generation, Linux integration, cross-compilation, and benchmarking into a single cohesive environment. The measured speedup confirms that even a small number of carefully designed, tightly coupled hardware instructions can deliver substantial gains for the class of workloads most relevant to embedded cryptography and machine learning. The open-source playground provides a reproducible platform for the community to build on, and the limitations identified here chart a clear path toward a more complete implementation.

Appendix A

Detailed FPGA Synthesis Area Reports

This appendix provides the per-instance cell counts produced by Vivado synthesis for each of the three CXU configurations evaluated in Chapter 5.

All figures are for the Digilent Arty S7-50 (Xilinx Spartan-7 XC7S50) target, synthesised with Vivado in out-of-context mode as part of the LiteX SoC build flow. The *Cells* column reports the number of primitive logic primitives (LUTs, flip-flops, carry chains, etc.) assigned to each hierarchical module after synthesis.

Table A.1: Per-module cell counts for the AES CXU configuration (total: 20 948 cells).

#	**Instance**	**Module**	**Cells**
1	top		20 948
2	Cxu0	Cxu0	209
3	VexiiRiscvLitex	VexiiRiscvLitex	12 967
4	cpuResetCtrl_fiber_aggregator_asyncBuffers_0	BufferCC	2
5	cpuResetCtrl_fiber_buffer	BufferCC_1	2
6	mem_toAxi4_logic_bridge	Axi4Bridge	401
7	a_ctx	ContextAsyncBufferFull	10
8	contexts	Ram_1w_1ra (p3)	10
9	d_arbiter	StreamArbiter_7	26
10	peripheral_bus_arbiter_core	Arbiter_1	91
11	a_arbiter	StreamArbiter_6	91
12	peripheral_bus_decoder_core	Decoder_1	72
13	d_arbiter	StreamArbiter_9	72
14	peripheral_clint_thread_core	TilelinkClint	362
15	peripheral_plic_intc_thread_logic	TilelinkPlic	410
16	peripheral_toAxiLite4_logic_bridge	AxiLite4Bridge	153
17	splited_mBus_arbiter_core	Arbiter	61
18	a_arbiter	StreamArbiter_5	61
19	splited_mBus_decoder_core	Decoder	18
20	d_arbiter	StreamArbiter_8	18
21	splited_mBus_to_mem_toAxi4_up_widthAdapter	WidthAdapter	246
22	upsize_d_ctx	ContextAsyncBufferFull_1	66
23	contexts	Ram_1w_1ra (p4)	66
24	splited_mBus_to_peripheral_bus_widthAdapter	WidthAdapter_1	103
25	vexiis_0_logic_core	VexiiRiscv	8 347
26	BtbPlugin_logic_ras_mem_stack	Ram_1w_1ra (p0)	19
27	CxuPlugin_logic_cxStateMem	RamSyncMwMux	211
28	location	RamAsyncMwReg	202
29	CxuPlugin_logic_onJoin_busRspStream_fifo	StreamFifoLowLatency	174
30	fifo	StreamFifo	174
31	logic_ram	Ram_1w_1ra	148
32	FetchL1Plugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1)	77
33	FetchL1Plugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_0)	88
34	FetchL1Plugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2)	31
35	LsuL1Plugin_logic_banks_0_mem	Ram_1w_1rs	71
36	LsuL1Plugin_logic_banks_1_mem	Ram_1w_1rs_1	13
37	LsuL1Plugin_logic_banks_2_mem	Ram_1w_1rs_2	13
38	LsuL1Plugin_logic_banks_3_mem	Ram_1w_1rs_3	101
39	LsuPlugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1_4)	19
40	LsuPlugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_5)	15
41	LsuPlugin_translationStorage_sl_0_ways_2	Ram_1w_1ra (p1_6)	27
42	LsuPlugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2_7)	22
43	early0_DivPlugin_logic_processing_div	DivRadix	554
44	integer_RegFilePlugin_logic_regfile_fpga	RegFileMem	307

Table A.2: Per-module cell counts for the GZIP CXU configuration (total: 20 666 cells).

#	**Instance**	**Module**	**Cells**
1	top		20 666
2	Cxu0	Cxu0	20
3	VexiiRiscvLitex	VexiiRiscvLitex	12 857
4	cpuResetCtrl_fiber_aggregator_asyncBuffers_0	BufferCC	2
5	cpuResetCtrl_fiber_buffer	BufferCC_1	2
6	mem_toAxi4_logic_bridge	Axi4Bridge	401
7	a_ctx	ContextAsyncBufferFull	10
8	contexts	Ram_1w_1ra (p3)	10
9	d_arbiter	StreamArbiter_7	26
10	peripheral_bus_arbiter_core	Arbiter_1	91
11	a_arbiter	StreamArbiter_6	91
12	peripheral_bus_decoder_core	Decoder_1	71
13	d_arbiter	StreamArbiter_9	71
14	peripheral_clint_thread_core	TilelinkClint	362
15	peripheral_plic_intc_thread_logic	TilelinkPlic	413
16	peripheral_toAxiLite4_logic_bridge	AxiLite4Bridge	152
17	splited_mBus_arbiter_core	Arbiter	61
18	a_arbiter	StreamArbiter_5	61
19	splited_mBus_decoder_core	Decoder	18
20	d_arbiter	StreamArbiter_8	18
21	splited_mBus_to_mem_toAxi4_up_widthAdapter	WidthAdapter	246
22	upsize_d_ctx	ContextAsyncBufferFull_1	66
23	contexts	Ram_1w_1ra (p4)	66
24	splited_mBus_to_peripheral_bus_widthAdapter	WidthAdapter_1	103
25	vexiis_0_logic_core	VexiiRiscv	8 271
26	BtbPlugin_logic_ras_mem_stack	Ram_1w_1ra (p0)	19
27	CxuPlugin_logic_cxStateMem	RamSyncMwMux	212
28	location	RamAsyncMwReg	204
29	CxuPlugin_logic_onJoin_busRspStream_fifo	StreamFifoLowLatency	270
30	fifo	StreamFifo	270
31	logic_ram	Ram_1w_1ra	132
32	FetchL1Plugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1)	76
33	FetchL1Plugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_0)	89
34	FetchL1Plugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2)	31
35	LsuL1Plugin_logic_banks_0_mem	Ram_1w_1rs	70
36	LsuL1Plugin_logic_banks_1_mem	Ram_1w_1rs_1	13
37	LsuL1Plugin_logic_banks_2_mem	Ram_1w_1rs_2	13
38	LsuL1Plugin_logic_banks_3_mem	Ram_1w_1rs_3	101
39	LsuPlugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1_4)	19
40	LsuPlugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_5)	15
41	LsuPlugin_translationStorage_sl_0_ways_2	Ram_1w_1ra (p1_6)	27
42	LsuPlugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2_7)	22
43	early0_DivPlugin_logic_processing_div	DivRadix	441
44	integer_RegFilePlugin_logic_regfile_fpga	RegFileMem	431

Table A.3: Per-module cell counts for the TFLite MAC CXU configuration (total: 20 698 cells). There is no separate `Cxu0` top-level instance; the MAC unit is optimized out for this specific instance.

#	**Instance**	**Module**	**Cells**
1	top		20 698
2	VexiiRiscvLitex	VexiiRiscvLitex	12 874
3	cpuResetCtrl_fiber_aggregator_asyncBuffers_0	BufferCC	2
4	cpuResetCtrl_fiber_buffer	BufferCC_1	2
5	mem_toAxi4_logic_bridge	Axi4Bridge	402
6	a_ctx	ContextAsyncBufferFull	10
7	contexts	Ram_1w_1ra (p3)	10
8	d_arbiter	StreamArbiter_7	26
9	peripheral_bus_arbiter_core	Arbiter_1	91
10	a_arbiter	StreamArbiter_6	91
11	peripheral_bus_decoder_core	Decoder_1	71
12	d_arbiter	StreamArbiter_9	71
13	peripheral_clint_thread_core	TilelinkClint	362
14	peripheral_plic_intc_thread_logic	TilelinkPlic	413
15	peripheral_toAxiLite4_logic_bridge	AxiLite4Bridge	152
16	splited_mBus_arbiter_core	Arbiter	61
17	a_arbiter	StreamArbiter_5	61
18	splited_mBus_decoder_core	Decoder	18
19	d_arbiter	StreamArbiter_8	18
20	splited_mBus_to_mem_toAxi4_up_widthAdapter	WidthAdapter	246
21	upsize_d_ctx	ContextAsyncBufferFull_1	66
22	contexts	Ram_1w_1ra (p4)	66
23	splited_mBus_to_peripheral_bus_widthAdapter	WidthAdapter_1	103
24	vexiis_0_logic_core	VexiiRiscv	8 288
25	BtbPlugin_logic_ras_mem_stack	Ram_1w_1ra (p0)	18
26	CxuPlugin_logic_cxStateMem	RamSyncMwMux	224
27	location	RamAsyncMwReg	216
28	CxuPlugin_logic_onJoin_busRspStream_fifo	StreamFifoLowLatency	162
29	fifo	StreamFifo	162
30	logic_ram	Ram_1w_1ra	127
31	FetchL1Plugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1)	83
32	FetchL1Plugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_0)	82
33	FetchL1Plugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2)	31
34	LsuL1Plugin_logic_banks_0_mem	Ram_1w_1rs	70
35	LsuL1Plugin_logic_banks_1_mem	Ram_1w_1rs_1	13
36	LsuL1Plugin_logic_banks_2_mem	Ram_1w_1rs_2	13
37	LsuL1Plugin_logic_banks_3_mem	Ram_1w_1rs_3	101
38	LsuPlugin_translationStorage_sl_0_ways_0	Ram_1w_1ra (p1_4)	19
39	LsuPlugin_translationStorage_sl_0_ways_1	Ram_1w_1ra (p1_5)	15
40	LsuPlugin_translationStorage_sl_0_ways_2	Ram_1w_1ra (p1_6)	27
41	LsuPlugin_translationStorage_sl_1_ways_0	Ram_1w_1ra (p2_7)	22
42	early0_DivPlugin_logic_processing_div	DivRadix	554
43	integer_RegFilePlugin_logic_regfile_fpga	RegFileMem	524

Bibliography

[1] N. Alawadi et al. *Microarchitecture Design & Benchmarking of Custom SHA-3 Instruction for RISC-V*. https://arxiv.org/abs/2508.20653. Implements the full Keccak-f permutation as a custom RISC-V instruction. 2025.

[2] Sebastien Bourdeauducq and contributors. *MiSoC: A Python-Based System-on-Chip Construction Framework*. https://github.com/m-labs/misoc. Predecessor of MIGen that pioneered Python-based SoC generation for FPGA platforms; now deprecated but historically significant. 2014.

[3] Songqiao Cui and Josep Balasch. "Efficient Software Masking of AES through Instruction Set Extensions". In: Introduces constant-time GF($2\hat{8}$) multiplication and addition instructions for masked AES, achieving speedups over software masking. 2023. URL: https://past.date-conference.com/proceedings-archive/2023/DATA/895.pdf.

[4] Robert David et al. "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems". In: *Proceedings of Machine Learning and Systems*. Vol. 3. 2021, pp. 800–811. URL: https://arxiv.org/pdf/2010.08678.

[5] Digilent Inc. *Arty S7 Reference Manual*. Accessed: 2025. Digilent Inc. 2023. URL: https://digilent.com/reference/programmable-logic/arty-s7/reference-manual.

[6] Florian Egert. "FRANCIS-V: Framework for Integrating Custom Instructions into RISC-V Systems". Framework integrating custom instruction generators with Core-V XIF, emphasizing tooling, CI, and reuse. MA thesis. TU Wien, 2023. URL: https://repositum.tuwien.at/bitstream/20.500.12708/188868/1/Egert%20Florian%20-%202023%20-%20FRANCIS-V%20FRAmework%20for%20iNtegrating%20Custom%20InstructionS...pdf.

[7] enjoy-digital. *LiteDram: Small footprint and configurable DRAM core*. https://github.com/enjoy-digital/litedram. Litex modules for DRAM interoperability. 2025.

[8] enjoy-digital. *LiteEth: Small footprint and configurable Ethernet core*. https://github.com/enjoy-digital/liteeth. Litex modules for Ethernet interoperability. 2025.

[9] enjoy-digital. *LitePcie: Small footprint and configurable PCIe core*. https://github.com/enjoy-digital/litepcie. Litex modules for PCIE interoperability. 2025.

[10] enjoy-digital. *LiteX: Build your hardware, easily!* https://github.com/enjoy-digital/litex. Open-source Python SoC builder and FPGA IP library for creating custom embedded systems. 2025.

[11] enjoy-digital. *LiteX: Build your hardware, easily!* https://github.com/litex-hub/linux-on-litex-vexriscv. Setup to run linux on a litex-based SoC. 2025.

[12] F4PGA Authors. *F4PGA: Fully Open-Source FPGA Toolchain*. https://github.com/chipsalliance/f4pga. Open-source FPGA synthesis, place-and-route, and bitstream generation framework, commonly used together with FPGA-based SoC frameworks such as LiteX. 2019.

[13] GNU Software foundation. *Sources and Website for the GZIP allication*. https://gzip.org/. 2025.

[14] RISC-V foundation. *Open Source Supervisor Binary Interface implementation*. https://github.com/riscv-software-src/opensbi. 2025.

[15] The linux foundation. *Documentation of the linux device tree*. https://www.kernel.org/doc/html/latest/devicetree/usage-model.html. 2025.

[16] Jan Gray et al. *CFU Specification*. https://cfu.readthedocs.io/en/latest/. Specification of the Composable Custom Function Unit (CFU); used for developing extensions and illustrating use cases. 2025.

[17] Jan Gray et al. *CXU Specification*. https://riscv.atlassian.net/wiki/spaces/CCEX/pages/179240966/Requirements. Specification of the Composable Custom Unit (CXU); used for developing extensions and illustrating use cases. 2025.

[18] Jan Gray et al. *Github sources for the CXU Specification*. https://github.com/riscv/composable-custom-extensions/tree/main. Github code for the specification of the Composable Custom Unit (CXU). 2025.

[19] Liangzhen Lai, Naveen Sze, et al. "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs". In: *arXiv preprint arXiv:1801.06601* (2018). URL: https://arxiv.org/pdf/1801.06601.

[20] Itamar Levi, Thomas Eisenbarth, et al. *RISQ-V: Tightly Coupled Accelerators for PQC on RISC-V*. https://eprint.iacr.org/2020/446. Introduces 29 new instructions for lattice-based PQC; reports up to 9.6× speedup for Kyber. 2020.

[21] Ben Marshall et al. "The Design of Scalar AES Instruction Set Extensions for RISC-V". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021.1 (2020), pp. 109–136. DOI: 10.46586/tches.v2021.i1.109-136. URL: https://tches.iacr.org/index.php/TCHES/article/view/8729.

[22] Petro Mozil. *CXU-playground: Experimental Accelerator Instruction and Core Extensions*. https://github.com/pmozil/CXU-playground. GitHub repository. 2026.

[23] NIST. *The AES encryption standard specification*. The description of the AES algorithm. URL: \url{https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf}.

[24] L. de Oliveira et al. *MiniFloat-NN & ExSdotp: Modular ISEs for Low-Precision NN Training on RISC-V*. https://arxiv.org/abs/2207.03192. Proposes modular RISC-V ISA extensions for float8 dot-product accumulation; suitable for CXU-style integration. 2022.

[25] OpenHW Group. *Core-V XIF*. https://github.com/openhwgroup/core-v-xif. Extension interface for RISC-V cores enabling custom instructions and CSRs; predecessor to CFU-style approaches. 2021.

[26] U-Boot organization. *Website for the universal bootloader*. https://u-boot.org/. 2025.

[27] Charles Papon and contributors. *SpinalHDL: A Scala-Based Hardware Description Language for Extensible Digital Design*. https://spinalhdl.github.io/SpinalDoc. Hardware construction language enabling highly parameterizable and extensible designs; widely used for building RISC-V SoCs such as VexRiscv. 2018.

[28] Charles Papon and contributors. *VexiiRiscv: Like VexRiscv, but, Harder, Better, Faster, Stronger*. https://github.com/SpinalHDL/VexiiRiscv. An updated version of VexRiscV witha clearer implementation and newer functionality.

[29] Shvetank Prakash et al. "CFU Playground: Full-Stack Open-Source Framework for Tiny Machine Learning (tinyML) Acceleration on FPGAs". In: *arXiv preprint* (2022). A full-stack open-source framework for rapid design and evaluation of custom function units (CFUs) to accelerate FPGA-based RISC-V soft CPUs for embedded machine learning applications. arXiv: 2201.01863 [cs.AR]. URL: https://arxiv.org/abs/2201.01863.

[30] R. Primas et al. "NLU-V: Instruction Set Extensions for Efficient Symmetric Cryptography on RISC-V". In: *Journal of Low Power Electronics and Applications* (2024). Introduces a non-linear + linear unit (NLU) extension for block ciphers such as AES and PRESENT. URL: https://www.mdpi.com/2410-387X/8/1/9.

[31] D. Rakhmatov et al. "RISC-V Instruction Set Architecture Extensions: A Survey". In: *IvySci* (2023). Comprehensive survey of RISC-V instruction set extensions. URL: https://www.ivysci.com/articles/8240939__RISCV_Instruction_Set_Architecture_Extensions_A_Survey.

[32] RISC-V International. *The RISC-V Instruction Set Architecture and Ratified Specifications*. https://riscv.org/specifications/ratified/. Ratified RISC-V ISA specifications maintained by RISC-V International, the open standard organization for the RISC-V ISA. RISC-V International is a nonprofit governing body coordinating development of the ISA and extensions. 2025.

[33] Davide Rossi et al. *PULP Platform: An Open Parallel Ultra-Low-Power RISC-V System-on-Chip*. https://pulp-platform.org. Open-source RISC-V SoC platform focused on energy-efficient parallel processing and accelerator-centric architectures. 2015.

[34] A. Sozbilir et al. "ISA Extensions for Finite Field Arithmetic Accelerating Kyber and NewHope on RISC-V". In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2020). Proposes a RISC-V custom ISA extension for finite-field arithmetic; prototyped on VexRiscv with up to 85% speedup. URL: http://doi.org/10.13154/tches.v2020.i3.219-242.

[35] RFC specification. *Definition of the DEFLATE algorithm*. https://www.rfc-editor.org/rfc/rfc1951. 2025.

[36] M. Vranken et al. "Uniform Instruction Set Extensions for Multiplications in Contemporary and Post-Quantum Cryptography". In: (2023). Proposes uniform ISA support for big-integer and polynomial multiplication, targeting ECC and lattice-based PQC on RISC-V.